Sistema de reservas aereas

Angel Santiago Diaz Rueda
Jefferson Gutierrez tovar

Big Data

U. Sergio Arboleda
Ciencias exactas e ingeniería

17 de mayo de 2026

Tabla de contenido

Tabla de contenido 2
0. Contexto y supuestos 2
1 Requisitos funcionales (FR) 5
2 Requisitos no funcionales (NFR) 9
3 Modelo entidad-relación 13
4 Arquitectura del backend 25
5 Arquitectura del frontend 31
6 Primera implementación del backend 36
7 Implementación AWS — ETL, Glue Data Catalog y costos 42
Apéndice — Registro de prompts (prompt log) 53

0. Contexto y supuestos

0.1 Escenario de negocio

Una aerolínea **regional** contrata al equipo para construir su sistema de reservas

de vuelos desde cero. Directrices del negocio (deliberadamente incompletas):

- Los pasajeros buscan vuelos, reservan y pagan billetes, y consultan o cancelan sus reservas.
- Venta **directa** (sitio web) y a través de **agencias de viajes asociadas**.
- **Tarifas** variables según antelación de compra y tipo de asiento (economía, negocios).
- Un vuelo puede tener **escalas**; un pasajero puede reservar un **itinerario multi-tramo en una sola transacción**.
- La gerencia comercial quiere analizar después: **ocupación por ruta**, **ingresos por tarifa**, **patrones de cancelación**.
- Regla dura: *"el sistema no debe vender más asientos de los que tiene la aeronave"*.
- *"Los picos de tráfico durante la temporada alta"* — sin cifras concretas.

0.2 Supuestos declarados

> *Un supuesto no declarado es una decisión arquitectónica implícita. Todos los supuestos siguientes son decisiones del equipo, revisables, y cada uno indica qué parte del diseño se vería afectada si cambiara.*

#	Supuesto	Justificación	Impacto si cambia
S1	**No hay programa de viajero frecuente** en el MVP.	Fuera del núcleo evaluado; añade entidades (cuenta de millas, niveles) sin afectar la regla de sobreventa.	Nuevas entidades `LoyaltyAccount`, `MileTransaction`; reglas de tarifa con redención.
S2	**Se permiten cancelaciones.** El **cambio de fecha** se modela como *cancelar + volver a reservar* (con política de penalización configurable).	Evita lógica de reasignación de inventario entre instancias de vuelo; mantiene el flujo de reserva atómico y simple.	Nuevo caso de uso "modificar reserva" con transacción que libera un `SeatInventory` y toma otro atómicamente.
S3	**Agencias asociadas: mismo flujo de reserva**, diferenciadas por `channel = AGENCY` y con **comisión** registrada por reserva. Sin	La regla de sobreventa y el modelo de datos son idénticos; la diferencia es	Se añade una API B2B con su propia autenticación y contrato; el backend monolítico expone un router adicional.

	API B2B separada en el MVP (se diseña, no se implementa).	comercial (comisión, descuento), no transaccional.	
S4	**Reserva sin pago = *hold* temporal de 15 minutos** sobre el inventario (estado `HELD`); si no se paga, expira y libera los asientos.	El enunciado pregunta explícitamente si se permite reservar sin pagar. El hold protege al pasajero durante el pago sin bloquear inventario indefinidamente.	Cambiar a "pago atómico" elimina el estado `HELD` y el job de expiración; simplifica pero empeora UX en pagos lentos.
S5	**Una sola moneda (COP) y un idioma (es-CO)** en el MVP. El diseño contempla i18n y multi-moneda (tarifas con `currency`, textos externalizados).	Reduce alcance sin comprometer el modelo (la columna `currency` ya existe).	Tabla de tipos de cambio, `Fare` por moneda, frontend con catálogo de idiomas.
S6	**Objetivo de disponibilidad: 99.9% mensual** (≤ ~43 min de caída/mes) como traducción de *"no puede caerse en temporada alta"*. MVP en single-AZ; el documento declara el trade-off de costo hacia Multi-AZ.	Número verificable y defendible; Multi-AZ duplica el costo de RDS.	Multi-AZ en ambas RDS, réplicas de lectura para búsqueda, mayor costo (ver 5.7).
S7	**Asignación de asiento nominal ocurre en el check-in**, no en la compra. En la compra solo se descuenta un **contador de inventario por cabina**.	Simplifica el bloqueo de concurrencia (un contador por cabina, no N filas de asiento); refleja la práctica real de aerolíneas regionales.	El inventario pasa a ser por asiento (`Seat` con estado); el lock afecta filas individuales; posible selección de asiento de pago.
S8	**Equipaje fuera de alcance** en el MVP (se puede añadir como `AncillaryService`).	No afecta la reserva ni la sobreventa.	Nueva entidad de servicios adicionales y su cobro.
S9	**Número de rutas: se asume del orden de 10–30 rutas** y crecimiento de +20/año (del enunciado). Volumen de reservas moderado (aerolínea regional).	Base para el dimensionamiento (instancias `db.t3.micro`) y la proyección de costos y del escenario ×10 en 5.7.	Re-dimensionar instancias, particionar `flight_instance`/`reservation` por fecha, revisar el cuello de botella del lock.

S10	**Pago delegado a un proveedor externo** (pasarela). El sistema **no almacena datos de tarjeta**; guarda solo un token/referencia y el estado del pago. En el MVP el proveedor es un *stub* simulado.	Saca el alcance PCI-DSS del sistema; el diseño de pagos real queda documentado.	Integración real con pasarela (webhooks de confirmación, conciliación, reintentos).

0.3 Actores del sistema

Todos operan sobre **el mismo backend** (un monolito modular), con **interfaces distintas** según el rol.

Actor	Descripción	Interfaz	Casos de uso principales
Pasajero	Cliente final que compra en el sitio web.	SPA pública	Buscar vuelos, reservar (incl. multi-tramo), pagar, consultar, cancelar.
Agente de agencia asociada	Empleado de una agencia de viajes que reserva a nombre de un pasajero.	SPA (vista de agente) / futura API B2B	Mismo flujo de reserva + registro de comisión y datos de la agencia.
Personal de aeropuerto	Realiza el check-in y la asignación de asiento.	Interfaz interna	Consultar reserva, asignar asiento (`SeatAssignment`), emitir tarjeta de embarque.
Administrador comercial	Gestiona el catálogo y consulta reportes.	Interfaz de administración	Alta/baja de rutas, vuelos programados, tarifas; consulta de reportes (ocupación, ingresos, cancelaciones) servidos desde la BD analítica.

0.4 Trazabilidad

Cada requisito de este documento y de los siguientes lleva un identificador (`FR-xx`, `NFR-xx`, entidad `E-xx`). Las decisiones de arquitectura (5.4, 5.5) y de implementación (5.6, 5.7) **citan explícitamente** el identificador del requisito que las origina, según exige la rúbrica de "Claridad y trazabilidad".

1 Requisitos funcionales (FR)

Formato: `ID | Descripción | Actor involucrado | Prioridad`.

*Prioridad: **Alta** (MVP, imprescindible) · **Media** (deseable en la entrega) · **Baja** (diseñado, no implementado en el MVP).*

Referencias: los actores están definidos en 00-contexto-supuestos.md §0.3. Los supuestos Sx también.

1.1 Decisiones sobre las preguntas guía

Pregunta guía del enunciado	Decisión del equipo	FR relacionados
¿Quiénes son los actores? ¿Mismo sistema o interfaces distintas?	4 actores (Pasajero, Agente de agencia, Personal de aeropuerto, Administrador) sobre **un mismo backend monolítico** con **interfaces distintas** por rol.	Todos
¿Reserva multi-tramo = una reserva con varios vuelos, o varias reservas encadenadas?	**Una sola `Reservation`** que contiene un **`Itinerary`** con varios **segmentos** (`ItinerarySegment`), cada uno ligado a una instancia de vuelo. La creación es **atómica**: o se confirman todos los segmentos o ninguno.	FR-04, FR-05
¿Se permite reservar sin pagar (hold), o el pago es atómico?	Se permite **hold temporal de 15 min** (estado `HELD`), luego expira y libera inventario (S4). El pago confirma la reserva (`CONFIRMED`).	FR-06, FR-07, FR-08
¿Qué pasa si el pasajero cancela o cambia de fecha?	**Cancelar**: libera inventario y registra evento + política de reembolso. **Cambio de fecha** = cancelar + reservar de nuevo, con penalización configurable (S2).	FR-09, FR-10
¿Las agencias tienen flujo distinto (descuento, comisión) o el mismo?	**Mismo flujo de reserva**; se marca `channel = AGENCY`, se aplica descuento/comisión configurable y se registra la agencia (S3). Sin API B2B separada en el MVP.	FR-04, FR-11

1.2 Tabla de requisitos funcionales

Búsqueda y catálogo

ID	Descripción	Actor	Prioridad
FR-01	El sistema permite buscar vuelos **directos** por origen, destino, fecha y número de pasajeros,	Pasajero, Agente de agencia	Alta

	mostrando horario, aeronave y tarifas disponibles por cabina.		
FR-02	El sistema permite buscar itinerarios **con escalas** (al menos 1 conexión), combinando instancias de vuelo cuyo tiempo de conexión sea válido (mínimo configurable).	Pasajero, Agente de agencia	Alta
FR-03	El sistema muestra, para cada resultado, el **desglose de tarifa** aplicable según la antelación de compra y la cabina (economía / negocios), y el precio total del itinerario.	Pasajero, Agente de agencia	Alta

Reserva y pago

ID	Descripción	Actor	Prioridad
FR-04	El sistema permite **crear una reserva** para uno o varios pasajeros sobre un itinerario de **uno o varios segmentos**, en **una sola transacción atómica**: si algún segmento no tiene inventario suficiente, no se crea ninguna parte de la reserva.	Pasajero, Agente de agencia	Alta
FR-05	Al crear la reserva, el sistema **descuenta el inventario** de asientos de cada segmento (contador por cabina, S7) garantizando que **nunca se vendan más asientos de los que tiene la aeronave**, incluso ante solicitudes simultáneas.	Pasajero, Agente de agencia	Alta
FR-06	El sistema permite crear la reserva en estado **`HELD`** (hold temporal de 15 min) sin pago inmediato; el inventario queda retenido durante ese tiempo.	Pasajero, Agente de agencia	Alta
FR-07	Si una reserva `HELD` no se paga antes de expirar, el sistema **libera automáticamente** el inventario retenido y marca la reserva como `EXPIRED`.	Sistema (job)	Alta
FR-08	El sistema permite **pagar** una reserva `HELD` a través de una pasarela externa; al confirmarse el pago la reserva pasa a **`CONFIRMED`** y se emiten los billetes (`Ticket`) por pasajero y segmento.	Pasajero, Agente de agencia	Alta
FR-09	El sistema permite **cancelar** una reserva `CONFIRMED` o `HELD`: libera el inventario, registra el evento y calcula el reembolso según la política vigente.	Pasajero, Agente de agencia	Alta
FR-10	El sistema permite **cambiar la fecha** de un itinerario ya confirmado como una operación de *cancelar + reservar* atómica, aplicando la	Pasajero, Agente de agencia	Media

	penalización configurada.		

Consulta

ID	Descripción	Actor	Prioridad
FR-12	El sistema permite **consultar una reserva** por su código, devolviendo estado, pasajeros, segmentos, tarifas, pagos e historial de cambios.	Pasajero, Agente de agencia, Personal de aeropuerto	Alta
FR-13	El sistema mantiene un **historial append-only** de todos los cambios de estado de una reserva (`ReservationEvent`) para auditoría.	Sistema	Alta

Canal de agencias

ID	Descripción	Actor	Prioridad
FR-11	El sistema registra, para cada reserva hecha por una agencia, la **agencia**, el **descuento** aplicado y la **comisión** devengada, sin alterar la regla de inventario.	Agente de agencia	Media

Check-in

ID	Descripción	Actor	Prioridad
FR-14	El sistema permite, en el **check-in**, asignar un asiento nominal (`SeatAssignment`) a un pasajero sobre un segmento confirmado, dentro de la cabina comprada.	Personal de aeropuerto	Baja

Administración y analítica

ID	Descripción	Actor	Prioridad
FR-15	El sistema permite al administrador gestionar el **catálogo**: aeropuertos, rutas, aeronaves y configuraciones de cabina.	Administrador	Media
FR-16	El sistema permite definir **vuelos programados** (`ScheduledFlight`: ruta, hora, días de la semana, aeronave) y **generar automáticamente** las instancias de vuelo (`FlightInstance`) con su	Administrador	Media

	inventario para un rango de fechas.		
FR-17	El sistema permite gestionar **tarifas** (`Fare`) por ruta, cabina y ventana de antelación.	Administrador	Media
FR-18	El sistema entrega a la gerencia comercial reportes de **ocupación por ruta**, **ingresos por tarifa** y **patrones de cancelación**, servidos desde la **base de datos analítica** alimentada por el proceso ETL (ver 07-aws-etl-glue-costos.md).	Administrador	Alta

1.3 Alcance de la implementación (5.6)

De la tabla anterior, la **primera implementación del backend** (sección 5.6) expone como mínimo: **FR-01/FR-02** (buscar vuelos), **FR-04/FR-05/FR-06** (crear reserva con control de concurrencia) y **FR-12** (consultar reserva). El resto queda diseñado y, según el tiempo, parcialmente implementado (FR-07, FR-08, FR-09 como operaciones de apoyo).

2 Requisitos no funcionales (NFR)

*Cada NFR tiene: **métrica o condición verificable**, el **riesgo de negocio** que mitiga, y **cómo se verifica**. No se admiten formulaciones tipo "debe ser rápido".*

Referencias a FR en 01-requisitos-funcionales.md; supuestos Sx en 00-contexto-supuestos.md.

2.1 Rendimiento

ID	Requisito	Métrica verificable	Riesgo de negocio	Verificación
NFR-01	Búsqueda de vuelos **directos** rápida.	Tiempo de respuesta **p95 < 800 ms** y p99 < 1.5 s, con la BD cargada con 30 rutas y 90 días de instancias de vuelo, hasta 50 búsquedas concurrentes.	Búsqueda lenta → abandono de la compra → pérdida de ingresos, sobre todo en temporada alta.	Prueba de carga (`locust`/`k6`) contra `GET /flights/search`; se registran percentiles.
NFR-02	Búsqueda de itinerarios **con escalas**	**p95 < 1.5 s** para itinerarios de hasta 1 conexión (el	Igual que NFR-01; además, timeouts en	Misma prueba de carga con parámetro de

	aceptablemente rápida pese al mayor costo de cómputo.	enunciado pregunta si cambia con escalas: **sí**, se admite ~2× el presupuesto de la búsqueda directa).	conexiones hacen que el usuario no vea rutas válidas.	conexiones; índices sobre `(route_id, departure_date)` y sobre aeropuertos de conexión.

Relación con la arquitectura: NFR-01/02 sustentan la elección de **SPA** (render en cliente, llamadas asíncronas a la API) y de índices específicos en la BD transaccional (ver 5.4/5.5).

2.2 Consistencia / Concurrencia — *núcleo de la evaluación*

ID	Requisito	Métrica / condición verificable	Riesgo de negocio	Verificación
NFR-03	**No sobreventa.** Si *N* solicitudes intentan reservar simultáneamente el/los último(s) asiento(s) de un segmento, el sistema confirma **exactamente** las que caben y **rechaza el resto con HTTP 409**; nunca `seats_sold > capacidad`.	Invariante tras cualquier ejecución concurrente: `seats_sold + seats_held ≤ capacity` para todo `SeatInventory`. Con *N* = 20 hilos compitiendo por 1 asiento → **1 confirmación, 19 rechazos, 0 sobreventa**.	Vender un asiento inexistente obliga a *denied boarding*: compensación económica al pasajero, coste de reacomodo y daño reputacional. Es la regla dura del enunciado.	`backend/tests/test_concurrencia.py`: lanza *N* clientes en paralelo contra `POST /reservations`; asserta 1 éxito, *N*−1 → 409, y consulta `SeatInventory` para verificar el invariante. Evidencia (log + captura) en 06-implementacion-backend.md.
NFR-04	**Atomicidad de la reserva multi-tramo (FR-04).** Una reserva de varios segmentos se confirma completa o no deja rastro.	Tras un fallo inyectado en el 2.º segmento, `Reservation`, `Itinerary`, `Ticket` y los descuentos de inventario del 1.er segmento **no existen** (rollback total).	Cobrar/reter inventario de un tramo sin el resto del viaje = pasajero varado, reembolsos manuales.	Test de integración con fallo simulado; se verifica el rollback en las 3 tablas y en el inventario.

Mecanismo elegido: bloqueo pesimista en PostgreSQL — dentro de la transacción de reserva se hace `SELECT ... FOR UPDATE` sobre la(s) fila(s) de `SeatInventory` de cada segmento, se valida la disponibilidad y se actualiza el contador antes del `COMMIT`. Justificación completa y alternativas descartadas (optimista, cola) en 04-arquitectura-backend.md.

2.3 Disponibilidad

ID	Requisito	Métrica verificable	Riesgo de negocio	Verificación
NFR-05	El sistema traduce *"no puede caerse en temporada alta"* como **99.9% de disponibilidad mensual** del servicio de reservas (≤ ~43 min de indisponibilidad al mes).	Uptime medido sobre el endpoint de salud durante un mes ≥ 99.9%. Implica: RDS con backups automáticos, y **camino a Multi-AZ** (RTO ≈ minutos) documentado como trade-off de costo.	Una caída durante un pico de ventas de temporada alta = ingresos perdidos irrecuperables (los asientos de esa fecha no se revenden luego).	En el MVP (Learner Lab) se despliega **single-AZ** por costo y se documenta explícitamente qué cuesta y qué implica pasar a Multi-AZ (ver proyección en 5.7). Se mide el uptime del endpoint de salud durante la demo.

99% (≈ 7.2 h/mes) se descarta: 7 horas de caída incluyen posibles picos completos. 99.99% (≈ 4 min/mes) exige multi-región y automatización de failover: sobredimensionado y fuera del presupuesto del Learner Lab. **99.9% es el punto de equilibrio** costo/complejidad para una aerolínea regional.

2.4 Seguridad

ID	Requisito	Métrica / condición verificable	Riesgo de negocio	Verificación
NFR-06	**Datos de pago fuera del sistema.** El sistema **no almacena PAN ni CVV**; solo guarda un token/referencia del proveedor y el estado del pago (S10).	Revisión del esquema: ninguna columna almacena datos de tarjeta. El flujo de pago se delega a la pasarela.	Almacenar datos de tarjeta implica cumplimiento **PCI-DSS** completo (auditorías, segmentación) y multas ante brecha.	Inspección del modelo de datos (5.3) y del código de pagos (5.6); prueba de que `POST /reservations/{code}/pay` solo maneja un token.
NFR-	**Protección de**	RDS con `StorageEncrypted`	Fuga de datos	Verificación de la

07	**datos personales (PII) de pasajeros.** Cifrado **en reposo** (RDS storage encryption / KMS) y **en tránsito** (TLS 1.2+ en la API). Acceso a PII restringido por rol.	`= true`; la API solo acepta HTTPS; los endpoints de consulta exigen el código de reserva + apellido (no listables).	personales → sanción bajo la Ley 1581 de 2012 (habeas data, Colombia) y pérdida de confianza.	configuración de RDS y del certificado TLS; prueba de que no existe endpoint que liste reservas o pasajeros sin credenciales.

2.5 Escalabilidad

ID	Requisito	Métrica / condición verificable	Riesgo de negocio	Verificación
NFR-08	El sistema soporta el crecimiento de **+20 rutas/año** (S9) sin rediseño. Se identifica que el **primer cuello de botella** es la **contención del bloqueo pesimista sobre `SeatInventory`** en vuelos muy demandados, no el número de rutas.	Con el doble de rutas y de tráfico, el p95 de `POST /reservations` se mantiene < 1.5 s. El lock se toma **por `flight_instance` + cabina** (fila específica), nunca global, de modo que reservas de vuelos distintos no se serializan entre sí.	Si toda reserva se serializara, en temporada alta se formaría una cola creciente → timeouts → ventas perdidas justo cuando más importa.	Prueba de carga con reservas repartidas entre muchos vuelos (contención baja) y concentradas en uno (contención alta); se comparan percentiles. Mitigaciones documentadas en 5.4: índice sobre `SeatInventory(flight_instance_id, cabin)`, transacción corta, y (a futuro) réplicas de lectura para la búsqueda.

2.6 Auditabilidad

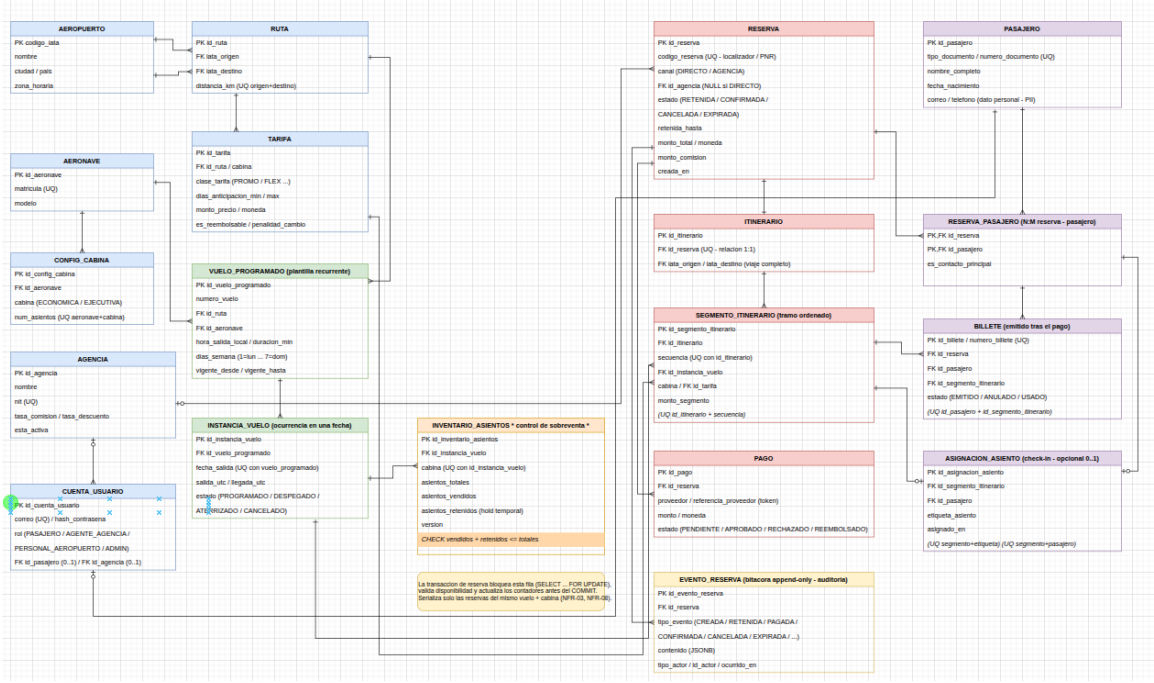
ID	Requisito	Métrica / condición verificable	Riesgo de negocio	Verificación
NFR-09	Se puede **reconstruir el**	Tabla `ReservationEvent`	Disputas con pasajeros o	Test que ejecuta un ciclo de vida

	historial completo de cambios de cualquier reserva (creación, hold, pago, cancelación, cambio de fecha, asignación de asiento) con marca de tiempo y actor.	**append-only** (sin `UPDATE`/`DELETE`); para toda reserva, la secuencia de eventos explica su estado actual. `GET /reservations/{code}` incluye el historial (FR-13).	agencias ("yo sí pagué", "nunca cancelé"), y necesidad de auditar las comisiones de agencia. Sin historial, cada disputa es palabra contra palabra.	completo de una reserva y verifica que cada transición generó su evento; intento de `UPDATE` sobre `ReservationEvent` debe fallar (permisos / trigger).

2.7 Resumen de trazabilidad NFR → decisión de arquitectura

NFR	Decisión que lo implementa	Documento
NFR-01, NFR-02	SPA + índices de búsqueda + (futuro) réplica de lectura	5.4, 5.5
NFR-03, NFR-04	Monolito con transacción ACID + `SELECT FOR UPDATE` sobre `SeatInventory`	5.4, 5.6
NFR-05	RDS con backups; trade-off single-AZ vs Multi-AZ documentado con costo	5.7
NFR-06, NFR-07	Pago delegado (sin datos de tarjeta) + RDS encryption + TLS + endpoints no listables	5.3, 5.4, 5.6
NFR-08	Lock por fila `flight_instance`+cabina, transacción corta, índice dedicado	5.4
NFR-09	`ReservationEvent` append-only	5.3, 5.6

3 Modelo entidad-relación



3.1 Decisiones de modelado (preguntas guía del enunciado)

Pregunta guía	Decisión	Justificación	Requisito que la origina
¿Itinerario multi-tramo: una reserva con muchos vuelos, o el itinerario es entidad propia?	**`ITINERARIO` es entidad propia** entre `RESERVA` y `INSTANCIA_VUELO`. Una reserva tiene **un** itinerario; el itinerario tiene **N `SEGMENTO_ITINERARIO`** ordenados, cada uno apuntando a una `INSTANCIA_VUELO` y a una `TARIFA`.	Un viaje multi-tramo es una unidad de negocio (precio total, reglas de conexión, se cancela junto). Modelarlo como entidad permite validar tiempos de conexión y precio agregado sin duplicar lógica en la reserva. Reservas encadenadas se descartan: romperían la atomicidad de FR-04.	FR-04, FR-05, NFR-04
¿La asignación de asiento es parte de la reserva desde la compra, o es un proceso aparte (check-in)?	**Proceso aparte.** En la compra solo se descuenta un **contador de inventario por cabina** (`INVENTARIO_ASIENTOS`). El asiento nominal se asigna en el check-in vía `ASIGNACION_ASIENTO` (0..1 por pasajero-segmento).	El bloqueo pesimista se toma sobre **una fila** (`INVENTARIO_ASIENTOS` del vuelo+cabina), no sobre N filas de asiento → menor contención (NFR-08). Refleja la práctica de aerolíneas regionales (asiento en el check-in). (Supuesto S7.)	NFR-03, NFR-08, S7
¿Cómo se	**Dos entidades:**	Separar plantilla de instancia	FR-16, FR-

representa que el mismo vuelo se repite cada día?	`VUELO_PROGRAMADO` (plantilla: ruta, hora de salida, días de la semana, aeronave, vigencia) y `INSTANCIA_VUELO` (ocurrencia en una **fecha concreta**, con estado y su inventario).	evita repetir datos por cada día y permite que inventario, tarifas efectivas y reservas cuelguen de la **instancia**. Generar instancias es un proceso batch (FR-16).	01
¿Dónde vive el control de inventario para evitar sobreventa?	En **`INVENTARIO_ASIENTOS`**: una fila por (`INSTANCIA_VUELO`, `cabina`) con `asientos_totales`, `asientos_vendidos`, `asientos_retenidos`. La disponibilidad es `asientos_totales – asientos_vendidos – asientos_retenidos`. Es la fila que se bloquea con `SELECT ... FOR UPDATE`.	Aislar el inventario en su propia tabla minimiza el alcance del lock y el tiempo de transacción (NFR-08). `asientos_totales` se copia de `CONFIG_CABINA` al crear la instancia, de modo que un cambio de aeronave futuro no altera vuelos ya vendidos.	NFR-03, NFR-04, NFR-08
¿Qué nivel de normalización, dado que alimenta el OLTP y el pipeline de Big Data?	**OLTP en 3FN** con una única desnormalización controlada: los contadores de `INVENTARIO_ASIENTOS` (`asientos_vendidos`, `asientos_retenidos`) son valores derivados que se mantienen transaccionalmente en vez de contar `BILLETE`s en cada reserva. El **OLAP** (5.7) usa un **esquema estrella** distinto, poblado por ETL.	3FN da integridad y evita anomalías en el transaccional (NFR-04, NFR-09). El contador derivado es la excepción justificada: contar billetes bajo lock en cada reserva escalaría mal (NFR-08). El análisis no se hace sobre el OLTP: se separa físicamente (5.7).	NFR-04, NFR-08, NFR-09, 5.7

3.2 Listado de entidades

#	Entidad	Rol
E-01	`AEROPUERTO`	Aeropuerto (código IATA, ciudad, país, zona horaria).
E-02	`RUTA`	Par origen-destino operado por la aerolínea.
E-03	`AERONAVE`	Aeronave física de la flota (matrícula, modelo).
E-04	`CONFIG_CABINA`	Nº de asientos por cabina (ECONOMICA / EJECUTIVA) para una aeronave.
E-05	`VUELO_PROGRAMADO`	Plantilla de vuelo recurrente (ruta, hora, días de semana, aeronave, vigencia).
E-06	`INSTANCIA_VUELO`	Ocurrencia de un vuelo programado en una fecha concreta; tiene estado.

E-07	INVENTARIO_ASIENTOS	Contador de inventario por (INSTANCIA_VUELO, cabina). **Punto de control de sobreventa.**
E-08	TARIFA	Tarifa por ruta, cabina y ventana de antelación de compra; incluye reglas de cambio/reembolso.
E-09	PASAJERO	Persona que viaja (documento, nombre, fecha de nacimiento, contacto).
E-10	CUENTA_USUARIO	Credenciales de acceso; puede pertenecer a un pasajero o a un agente de agencia.
E-11	AGENCIA	Agencia de viajes asociada (nombre, NIT, % comisión, % descuento).
E-12	RESERVA	Reserva: código, canal (DIRECTO/AGENCIA), estado, importe total, agencia (si aplica).
E-13	RESERVA_PASAJERO	Pasajeros incluidos en una reserva (N:M reserva-pasajero).
E-14	ITINERARIO	Itinerario de la reserva (unidad de viaje); 1:1 con RESERVA.
E-15	SEGMENTO_ITINERARIO	Tramo ordenado del itinerario: apunta a una INSTANCIA_VUELO, una TARIFA y una cabina.
E-16	BILLETE	Billete emitido por (RESERVA_PASAJERO, SEGMENTO_ITINERARIO) tras el pago.
E-17	PAGO	Intento/registro de pago de una reserva (proveedor externo, token, estado, importe).
E-18	ASIGNACION_ASIENTO	Asiento nominal asignado a un pasajero en un segmento, durante el check-in (opcional).
E-19	EVENTO_RESERVA	Bitácora **append-only** de cambios de estado de una reserva (auditoría).

3.3 Relaciones principales

- RUTA **origen** → AEROPUERTO (N:1) · RUTA **destino** → AEROPUERTO (N:1).
- AERONAVE → CONFIG_CABINA (1:N) — configuración de cabinas de la aeronave.
- VUELO_PROGRAMADO → RUTA (N:1) · → AERONAVE (N:1).
- INSTANCIA_VUELO → VUELO_PROGRAMADO (N:1).
- INSTANCIA_VUELO → INVENTARIO_ASIENTOS (1:N, una por cabina).
- TARIFA → RUTA (N:1).
- RESERVA → ITINERARIO (1:1) · → AGENCIA (N:0..1) · → PAGO (1:N) · → EVENTO_RESERVA (1:N).
- RESERVA ↔ PASAJERO vía RESERVA_PASAJERO (N:M).
- ITINERARIO → SEGMENTO_ITINERARIO (1:N, ordenados por secuencia).
- SEGMENTO_ITINERARIO → INSTANCIA_VUELO (N:1) · → TARIFA (N:1).
- BILLETE → RESERVA_PASAJERO (N:1) · → SEGMENTO_ITINERARIO (N:1).
- ASIGNACION_ASIENTO → RESERVA_PASAJERO (N:1) · → SEGMENTO_ITINERARIO (N:1); 0..1 por par.
- CUENTA_USUARIO → PASAJERO (0..1) · → AGENCIA (0..1).

3.4 Diccionario de datos

E-01 `AEROPUERTO`

Atributo	Tipo	Restr.	Descripción
codigo_iata	CHAR(3)	PK	Código IATA (p. ej. BOG).
nombre	VARCHAR(120)	NN	Nombre del aeropuerto.
ciudad	VARCHAR(80)	NN	Ciudad.
pais	CHAR(2)	NN	ISO 3166-1 alfa-2.
zona_horaria	VARCHAR(40)	NN	Zona horaria IANA (para calcular conexiones).

E-02 `RUTA`

Atributo	Tipo	Restr.	Descripción
id_ruta	BIGSERIAL	PK	Identificador.
iata_origen	CHAR(3)	FK→AEROPUERTO, NN	Origen.
iata_destino	CHAR(3)	FK→AEROPUERTO, NN	Destino.
distancia_km	INTEGER		Distancia (para reportes/combustible).
—		UQ(iata_origen,iata_destino)	No se repite una ruta.

E-03 `AERONAVE`

Atributo	Tipo	Restr.	Descripción
id_aeronave	BIGSERIAL	PK	Identificador.
matricula	VARCHAR(10)	NN, UQ	Matrícula.
modelo	VARCHAR(40)	NN	Modelo (p. ej. ATR 72).

E-04 `CONFIG_CABINA`

Atributo	Tipo	Restr.	Descripción
id_config_cabina	BIGSERIAL	PK	Identificador.
id_aeronave	BIGINT	FK→AERONAVE, NN	Aeronave.
cabina	VARCHAR(10)	NN, CHECK in (ECONOMICA, EJECUTIVA)	Cabina.
num_asientos	SMALLINT	NN, CHECK > 0	Nº de asientos de esa cabina.

—		UQ(id_aeronave,cabina)	Una config por cabina y aeronave.

E-05 `VUELO_PROGRAMADO`

Atributo	Tipo	Restr.	Descripción
id_vuelo_programado	BIGSERIAL	PK	Identificador.
numero_vuelo	VARCHAR(6)	NN	Nº de vuelo comercial (p. ej. RA123).
id_ruta	BIGINT	FK→RUTA, NN	Ruta operada.
id_aeronave	BIGINT	FK→AERONAVE, NN	Aeronave asignada por defecto.
hora_salida_local	TIME	NN	Hora de salida (local al origen).
duracion_min	SMALLINT	NN	Duración estimada.
dias_semana	SMALLINT[]	NN	Días en que opera (1=lun … 7=dom).
vigente_desde / vigente_hasta	DATE	NN	Vigencia de la programación.

E-06 `INSTANCIA_VUELO`

Atributo	Tipo	Restr.	Descripción
id_instancia_vuelo	BIGSERIAL	PK	Identificador.
id_vuelo_programado	BIGINT	FK→VUELO_PROGRAMADO, NN	Plantilla de origen.
fecha_salida	DATE	NN	Fecha de salida (local al origen).
salida_utc / llegada_utc	TIMESTAMPTZ	NN	Instantes reales (para conexiones y reportes).
estado	VARCHAR(12)	NN, default PROGRAMADO	PROGRAMADO / DESPEGADO / ATERRIZADO / CANCELADO.
—		UQ(id_vuelo_programado,fecha_salida)	Una instancia por fecha.

E-07 `INVENTARIO_ASIENTOS` *(control de sobreventa — NFR-03)*

Atributo	Tipo	Restr.	Descripción
id_inventario_asientos	BIGSERIAL	PK	Identificador.
id_instancia_vuelo	BIGINT	FK→INSTANCIA_VUELO, NN	Vuelo.
cabina	VARCHAR(10)	NN	Cabina.
asientos_totales	SMALLINT	NN, CHECK ≥ 0	Capacidad (copiada de CONFIG_CABINA al crear la instancia).
asientos_vendidos	SMALLINT	NN, default 0, CHECK ≥ 0	Asientos confirmados (pagados).
asientos_retenidos	SMALLINT	NN, default 0, CHECK ≥ 0	Asientos en hold temporal.
version	INTEGER	NN, default 0	Reservado por si se añade control optimista.
—		UQ(id_instancia_vuelo,cabina)	Una fila por vuelo y cabina.
—		CHECK (asientos_vendidos + asientos_retenidos ≤ asientos_totales)	**Invariante de no sobreventa a nivel de BD.**

E-08 `TARIFA`

Atributo	Tipo	Restr.	Descripción
id_tarifa	BIGSERIAL	PK	Identificador.
id_ruta	BIGINT	FK→RUTA, NN	Ruta.
cabina	VARCHAR(10)	NN	Cabina.
clase_tarifa	VARCHAR(20)	NN	Nombre comercial (p. ej. PROMO, FLEX).
dias_anticipacion_min / dias_anticipacion_max	SMALLINT	NN	Ventana de antelación de compra en que aplica.
monto_precio	NUMERIC(12,2)	NN, CHECK ≥ 0	Precio.
moneda	CHAR(3)	NN, default 'COP'	Moneda (S5).
es_reembolsable	BOOLEAN	NN	Reembolsable.
penalidad_cambio	NUMERIC(12,2)	NN, default 0	Penalización por cambio de fecha (FR-10).

E-09 `PASAJERO`

Atributo	Tipo	Restr.	Descripción
id_pasajero	BIGSERIAL	PK	Identificador.
tipo_documento	VARCHAR(10)	NN	CC / PA / etc.
numero_documento	VARCHAR(30)	NN	Número de documento (dato personal — NFR-07).
nombre_completo	VARCHAR(160)	NN	Nombre completo.
fecha_nacimiento	DATE		Fecha de nacimiento.
correo / telefono	VARCHAR(120)		Contacto (dato personal).
—		UQ(tipo_documento,numero_documento)	Documento único.

E-10 `CUENTA_USUARIO`

Atributo	Tipo	Restr.	Descripción
id_cuenta_usuario	BIGSERIAL	PK	Identificador.
correo	VARCHAR(120)	NN, UQ	Login.
hash_contrasena	VARCHAR(255)	NN	Hash (nunca en claro — NFR-07).
rol	VARCHAR(20)	NN	PASAJERO / AGENTE_AGENCIA / PERSONAL_AEROPUERTO / ADMIN.
id_pasajero	BIGINT	FK→PASAJERO, NULL	Si es un pasajero registrado.
id_agencia	BIGINT	FK→AGENCIA, NULL	Si es un agente de agencia.

E-11 `AGENCIA`

Atributo	Tipo	Restr.	Descripción
id_agencia	BIGSERIAL	PK	Identificador.
nombre	VARCHAR(120)	NN	Razón social.
nit	VARCHAR(20)	NN, UQ	NIT.
tasa_comision	NUMERIC(5,4)	NN, default 0	% comisión sobre la venta (FR-11).
tasa_descuento	NUMERIC(5,4)	NN, default 0	% descuento aplicado al pasajero.
esta_activa	BOOLEAN	NN, default true	

E-12 `RESERVA`

Atributo	Tipo	Restr.	Descripción
id_reserva	BIGSERIAL	PK	Identificador interno.
codigo_reserva	CHAR(6)	NN, UQ	Localizador (PNR) para consulta (FR-12).
canal	VARCHAR(10)	NN, CHECK in (DIRECTO, AGENCIA)	Canal de venta.
id_agencia	BIGINT	FK→AGENCIA, NULL	Agencia (si canal = AGENCIA).
estado	VARCHAR(12)	NN	RETENIDA / CONFIRMADA / CANCELADA / EXPIRADA.
retenida_hasta	TIMESTAMPTZ	NULL	Expiración del hold (FR-07).
monto_total	NUMERIC(12,2)	NN	Importe total del itinerario.
moneda	CHAR(3)	NN, default 'COP'	
monto_comision	NUMERIC(12,2)	NN, default 0	Comisión devengada (FR-11).
creada_en	TIMESTAMPTZ	NN, default now()	

E-13 `RESERVA_PASAJERO`

Atributo	Tipo	Restr.	Descripción
id_reserva	BIGINT	PK, FK→RESERVA	
id_pasajero	BIGINT	PK, FK→PASAJERO	
es_contacto_principal	BOOLEAN	NN, default false	Titular de la reserva.

E-14 `ITINERARIO`

Atributo	Tipo	Restr.	Descripción
id_itinerario	BIGSERIAL	PK	Identificador.
id_reserva	BIGINT	FK→RESERVA, NN, UQ	1:1 con la reserva.
iata_origen / iata_destino	CHAR(3)	FK→AEROPUERTO, NN	Origen/destino del viaje completo.

E-15 `SEGMENTO_ITINERARIO`

Atributo	Tipo	Restr.	Descripción

id_segmento_itinerario	BIGSERIAL	PK	Identificador.
id_itinerario	BIGINT	FK→ITINERARIO, NN	Itinerario padre.
secuencia	SMALLINT	NN	Orden del tramo (1, 2, …).
id_instancia_vuelo	BIGINT	FK→INSTANCIA_VUELO, NN	Vuelo del tramo.
cabina	VARCHAR(10)	NN	Cabina comprada.
id_tarifa	BIGINT	FK→TARIFA, NN	Tarifa aplicada.
monto_segmento	NUMERIC(12,2)	NN	Precio del tramo (× nº de pasajeros se agrega en la reserva).
—		UQ(id_itinerario,secuencia)	Secuencia sin huecos ni repetición.

E-16 `BILLETE`

Atributo	Tipo	Restr.	Descripción
id_billete	BIGSERIAL	PK	Identificador.
numero_billete	VARCHAR(15)	NN, UQ	Nº de billete (emitido tras el pago).
id_reserva	BIGINT	FK, NN	Reserva.
id_pasajero	BIGINT	FK, NN	Pasajero.
id_segmento_itinerario	BIGINT	FK, NN	Tramo.
estado	VARCHAR(12)	NN	EMITIDO / ANULADO / USADO.
—		UQ(id_pasajero,id_segmento_itinerario)	Un billete por pasajero y tramo.

E-17 `PAGO`

Atributo	Tipo	Restr.	Descripción
id_pago	BIGSERIAL	PK	Identificador.
id_reserva	BIGINT	FK→RESERVA, NN	Reserva pagada.
proveedor	VARCHAR(30)	NN	Pasarela externa (S10).
referencia_proveedor	VARCHAR(80)	NN	Token/referencia del proveedor (**no** datos de tarjeta — NFR-06).
monto	NUMERIC(12,2)	NN	Importe.
moneda	CHAR(3)	NN	
estado	VARCHAR(12)	NN	PENDIENTE / APROBADO / RECHAZADO / REEMBOLSADO.
creado_en	TIMESTAMPTZ	NN, default now()	

E-18 `ASIGNACION_ASIENTO` *(check-in — FR-14)*

Atributo	Tipo	Restr.	Descripción
id_asignacion_asiento	BIGSERIAL	PK	Identificador.
id_segmento_itinerario	BIGINT	FK, NN	Tramo.
id_pasajero	BIGINT	FK, NN	Pasajero.
etiqueta_asiento	VARCHAR(4)	NN	Asiento (p. ej. 14C).
asignado_en	TIMESTAMPTZ	NN, default now()	
—		UQ(id_segmento_itinerario,etiqueta_asiento)	Un asiento no se asigna dos veces.
—		UQ(id_segmento_itinerario,id_pasajero)	Un asiento por pasajero y tramo.

E-19 `EVENTO_RESERVA` *(auditoría append-only — NFR-09)*

Atributo	Tipo	Restr.	Descripción
id_evento_reserva	BIGSERIAL	PK	Identificador.
id_reserva	BIGINT	FK→RESERVA, NN	Reserva.

`tipo_evento`	VARCHAR(24)	NN	CREADA / RETENIDA / PAGADA / CONFIRMADA / CANCELADA / EXPIRADA / CAMBIO_FECHA / ASIENTO_ASIGNADO.
`contenido`	JSONB		Detalle del cambio (estado anterior/nuevo, importes).
`tipo_actor`	VARCHAR(20)	NN	PASAJERO / AGENTE_AGENCIA / PERSONAL_AEROPUERTO / SISTEMA.
`id_actor`	BIGINT		`id_cuenta_usuario` si aplica.
`ocurrido_en`	TIMESTAMPTZ	NN, default now()	
—		Sin UPDATE/DELETE (permisos de rol + trigger que rechaza modificación).	

5.3.5 Índices relevantes (rendimiento y concurrencia)

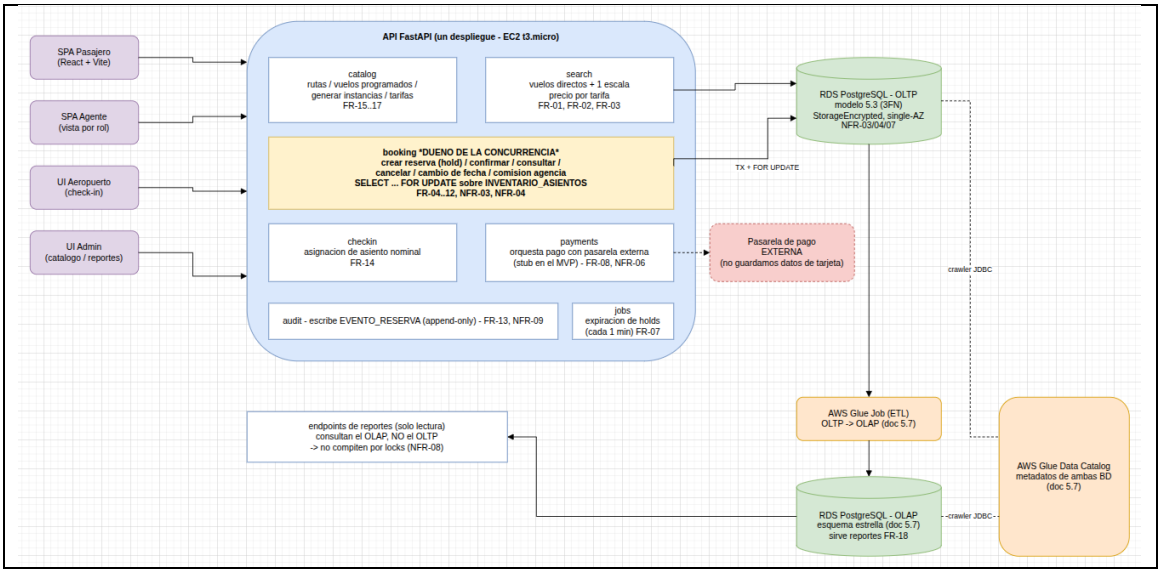
Índice	Motivo	NFR
`INSTANCIA_VUELO(id_vuelo_programado, fecha_salida)` **(UQ)**	Localizar instancias por fecha en la búsqueda.	NFR-01
`INSTANCIA_VUELO(salida_utc)`	Filtrar por fecha/hora y calcular conexiones.	NFR-02
`INVENTARIO_ASIENTOS(id_instancia_vuelo, cabina)` **(UQ)**	Fila exacta a bloquear con `SELECT ... FOR UPDATE`; lock puntual, no de tabla.	NFR-03, NFR-08
`VUELO_PROGRAMADO(id_ruta)`	Unir ruta→vuelos en la búsqueda.	NFR-01
`RESERVA(codigo_reserva)` **(UQ)**	Consulta por localizador.	FR-12
`EVENTO_RESERVA(id_reserva, ocurrido_en)`	Reconstruir historial ordenado.	NFR-09

5.3.6 Cómo el modelo previene la sobreventa (resumen)

1. Cada `INSTANCIA_VUELO` tiene una fila `INVENTARIO_ASIENTOS` por cabina con `asientos_totales` fijado al crear la instancia.
2. La transacción de reserva (5.4/5.6) ejecuta `SELECT ... FOR UPDATE` sobre esa(s) fila(s) → **serializa** las reservas que compiten por el mismo vuelo+cabina.

3. Valida `asientos_totales – asientos_vendidos – asientos_retenidos ≥ nº_pasajeros`; si no, aborta con 409 (NFR-03).
4. Incrementa `asientos_retenidos` (hold) o `asientos_vendidos` (pago), y el `CHECK (asientos_vendidos + asientos_retenidos ≤ asientos_totales)` actúa como **última barrera** a nivel de motor.
5. El job de expiración (FR-07) decrementa `asientos_retenidos` de holds vencidos, también bajo lock.

4 Arquitectura del backend



4.1 Estilo arquitectónico: monolito modular

Decisión. Un **único servicio** FastAPI (un despliegue, una base de datos transaccional) organizado en **módulos internos** con fronteras claras:

Módulo	Responsabilidad	FR que cubre
`catalog`	Aeropuertos, rutas, aeronaves, vuelos programados, generación de instancias, tarifas.	FR-15, FR-16, FR-17
`search`	Búsqueda de vuelos directos y con escalas, cálculo de precio por tarifa.	FR-01, FR-02, FR-03
`booking`	Creación de reserva (hold), confirmación, consulta, cancelación, cambio de fecha, comisión de agencia. **Dueño del control de concurrencia.**	FR-04 … FR-12, FR-11
`payments`	Orquestación del pago contra la pasarela externa (stub en el MVP).	FR-08, FR-09 (reembolso)
`checkin`	Asignación de asiento nominal.	FR-14

`audit`	Escritura de `EVENTO_RESERVA` (append-only) — invocado por `booking` y `checkin`.	FR-13, NFR-09
`jobs`	Tarea periódica de expiración de holds.	FR-07

Justificación (trazable a NFR).

- **NFR-03 / NFR-04 (no sobreventa, atomicidad multi-tramo).** El invariante de inventario y la reserva multi-segmento se resuelven con **una transacción ACID local** (`SELECT ... FOR UPDATE` + `COMMIT`). Con microservicios separados (búsqueda / reservas / pagos con bases de datos propias) haría falta una **saga distribuida** con compensaciones para lograr lo mismo: más complejidad, más modos de fallo, y una ventana en la que el inventario puede quedar inconsistente. Para una aerolínea **regional** (S9: ~10–30 rutas) el monolito es suficiente y correcto.
- **NFR-08 (escalabilidad).** El primer cuello de botella identificado no es el número de rutas sino la **contención del lock sobre `INVENTARIO_ASIENTOS`** en vuelos concretos. Separar en servicios no elimina esa contención (el lock sigue siendo por fila de inventario); solo añade latencia de red. La mitigación real —lock por `(id_instancia_vuelo, cabina)`, transacción corta, índice dedicado— vive dentro del monolito.
- **Claridad y trazabilidad (rúbrica).** Módulos con fronteras explícitas permiten, si el negocio crece, extraer `search` o `payments` a un servicio propio sin reescribir el núcleo. La decisión es reversible.

Alternativa considerada y descartada: microservicios desde el inicio (search-service, booking-service, payment-service). Descartada por el coste de la saga para garantizar NFR-03/04 y por el presupuesto del Learner Lab (más instancias, más red). Se documenta como camino de evolución.

4.2 Control de concurrencia contra la sobreventa

Decisión: bloqueo pesimista en PostgreSQL — `SELECT ... FOR UPDATE` sobre la(s) fila(s) de `INVENTARIO_ASIENTOS` implicadas, dentro de la transacción de reserva.

Flujo de `POST /reservations` (crea reserva en estado `RETENIDA`)

```
BEGIN;  -- isolation level READ COMMITTED (suficiente con FOR UPDATE)

-- 1. Bloquear el inventario de CADA segmento del itinerario, en orden
--    determinista (por id_instancia_vuelo asc) para evitar deadlocks.
SELECT id_inventario_asientos, asientos_totales, asientos_vendidos,
asientos_retenidos
  FROM inventario_asientos
 WHERE id_instancia_vuelo = ANY(:ids) AND cabina = :cabina
 ORDER BY id_instancia_vuelo
   FOR UPDATE;                      -- <-- serializa a los competidores por esas
filas

-- 2. Validar disponibilidad de cada fila
--    asientos_totales - asientos_vendidos - asientos_retenidos >= :num_pasajeros
--    Si alguna falla -> ROLLBACK y responder HTTP 409 (NFR-03).

-- 3. Aplicar el hold en cada fila
UPDATE inventario_asientos
   SET asientos_retenidos = asientos_retenidos + :num_pasajeros
 WHERE id_inventario_asientos = ANY(:locked_ids);
--    El CHECK (vendidos + retenidos <= totales) es la ultima barrera del motor.

-- 4. Crear RESERVA (estado=RETENIDA, retenida_hasta = now() + interval '15 min'),
--    ITINERARIO, SEGMENTO_ITINERARIO (x N), RESERVA_PASAJERO (x M).
-- 5. Insertar EVENTO_RESERVA (tipo_evento = 'CREADA' / 'RETENIDA').  (NFR-09)

COMMIT;   -- libera los locks
```

- **Orden determinista de bloqueo** (por `id_instancia_vuelo`): dos reservas multi-tramo que compartan dos vuelos en orden distinto no se bloquean mutuamente (previene deadlock).
- **Transacción corta**: entre `BEGIN` y `COMMIT` no hay llamadas externas (el pago es un paso posterior). Minimiza el tiempo de lock → menos contención (NFR-08).

- **Confirmación (`POST /reservations/{code}/pay`)**: al aprobar el pago,

transacción que hace `FOR UPDATE` de las mismas filas, mueve

`asientos_retenidos → asientos_vendidos`, pone `RESERVA.estado = CONFIRMADA`,

emite `BILLETE` por pasajero-segmento y registra el evento.

- **Expiración (`jobs`, FR-07)**: cada minuto, por cada reserva `RETENIDA` con

`retenida_hasta < now()`, transacción con `FOR UPDATE` que decrementa

`asientos_retenidos` y marca `EXPIRED` + evento.

Alternativas consideradas

Mecanismo	Por qué NO se eligió
Bloqueo optimista (columna `version`, `UPDATE ... WHERE version = X`, reintento en conflicto)	Funciona, y la columna `version` ya existe en el modelo por si se necesita. Se descartó para el MVP porque **bajo alta contención** (el escenario del enunciado: todos peleando por el último asiento en temporada alta) degenera en muchos reintentos y retrasa la respuesta justo en el peor momento. El pesimista da un comportamiento más predecible y es **más fácil de demostrar** en la prueba de concurrencia de 5.6.
Cola que serializa por vuelo (SQS + worker, una reserva a la vez por `id_instancia_vuelo`)	Serializa correctamente, pero introduce **procesamiento asíncrono**: el usuario no obtiene respuesta inmediata de si su reserva se confirmó, hace falta polling o webhooks, y añade piezas de infraestructura (cola, worker, dead-letter) difíciles de justificar en el Learner Lab para un MVP. Es la evolución natural si el volumen crece mucho (NFR-08), y se documenta como tal.

Cómo se verifica (enlace a 5.6): `backend/tests/test_concurrencia.py` lanza

N clientes concurrentes contra `POST /reservations` por el último asiento y

comprueba: 1 éxito, N−1 respuestas 409, e invariante

`asientos_vendidos + asientos_retenidos ≤ asientos_totales`.

4.3 Búsqueda de vuelos (FR-01, FR-02, NFR-01, NFR-02)

- **Directos (FR-01):** consulta sobre `INSTANCIA_VUELO ⋈ VUELO_PROGRAMADO ⋈ RUTA`

filtrando por origen, destino y `fecha_salida`, con `LEFT JOIN` a

`INVENTARIO_ASIENTOS` para mostrar disponibilidad y a `TARIFA` para el precio

según `dias_anticipacion`. La disponibilidad mostrada es **orientativa** (no se toma lock en la búsqueda); el lock real ocurre al reservar.

- **Con escalas (FR-02):** el módulo `search` compone hasta **1 conexión**: busca tramo A→X y X→B tales que `salida_utc(X→B) − llegada_utc(A→X)` esté entre un mínimo y un máximo de conexión configurables. Se limita a 1 escala en el MVP (para una aerolínea regional cubre la mayoría de itinerarios) y se admite un presupuesto de latencia ~2× el de la búsqueda directa (NFR-02).
- **Índices que lo sostienen** (definidos en 5.3.5): `INSTANCIA_VUELO(id_vuelo_programado, fecha_salida)`, `INSTANCIA_VUELO(salida_utc)`, `VUELO_PROGRAMADO(id_ruta)`.
- **Evolución (NFR-01/08):** si la búsqueda se vuelve el cuello de botella, se añade una **réplica de lectura** de la BD transaccional y `search` la consulta a ella, dejando la primaria para las escrituras de `booking`.

4.4 Integración del pago (FR-08, NFR-06)

Decisión: pago delegado a una pasarela externa. El sistema **no procesa ni almacena** datos de tarjeta (S10, NFR-06); solo guarda en `PAGO` el `proveedor`, la `referencia_proveedor` (token) y el `estado`.

Flujo:

6. El frontend obtiene del proveedor un token de pago (fuera de nuestro backend).
7. `POST /reservations/{code}/pay` con `{ token }` → el backend llama a la API del proveedor para **capturar** el cobro por `RESERVA.monto_total`.
8. Según la respuesta: `APROBADO` → transacción de confirmación (5.4.2); `RECHAZADO` → `PAGO.estado = RECHAZADO`, la reserva sigue `RETENIDA` hasta expirar.
9. Cancelación con reembolso (FR-09): `payments` solicita el `REFUND` al proveedor

y registra `PAGO.estado = REEMBOLSADO`.

- **MVP:** el proveedor es un **stub** (`payments/provider_stub.py`) que simula aprobado/rechazado de forma determinista para poder probar los flujos y la concurrencia sin depender de un tercero. El contrato de la interfaz (`PaymentProvider`) es el mismo que usaría una pasarela real → sustituible sin tocar `booking`.

4.5 Exposición a agencias asociadas (FR-11, S3)

Decisión (MVP): misma API que el pasajero final, diferenciada por el token de autenticación:

- Una `CUENTA_USUARIO` con `rol = AGENTE_AGENCIA` y `id_agencia` asociado.
- `booking` detecta el rol: marca `RESERVA.canal = AGENCIA`, aplica `AGENCIA.tasa_descuento` al precio y calcula `RESERVA.monto_comision` con `AGENCIA.tasa_comision`. La **regla de inventario es idéntica** (mismo `FOR UPDATE`, misma validación).
- **API B2B separada:** se **diseña** (endpoints de tarifa negociada, facturación consolidada, límites por agencia) pero **no se implementa** en el MVP. Se expone como router aparte (`/b2b/...`) el día que exista, reutilizando el mismo módulo `booking`.

4.6 Relación con el pipeline de Big Data del curso (5.7)

La arquitectura transaccional **no ejecuta análisis**. La relación con el pipeline es de **origen de datos**:

```
  [ OLTP PostgreSQL ]  --(vistas de extraccion)-->  [ AWS Glue Job / ETL ]  -->  [ OLAP PostgreSQL (estrella) ]
        (este doc)                                          (doc 5.7)                (doc 5.7)
                                                                 |
                                                  ambos registrados en  [ AWS Glue Data Catalog ]
```

- El backend expone, además, endpoints de solo lectura para los **reportes**

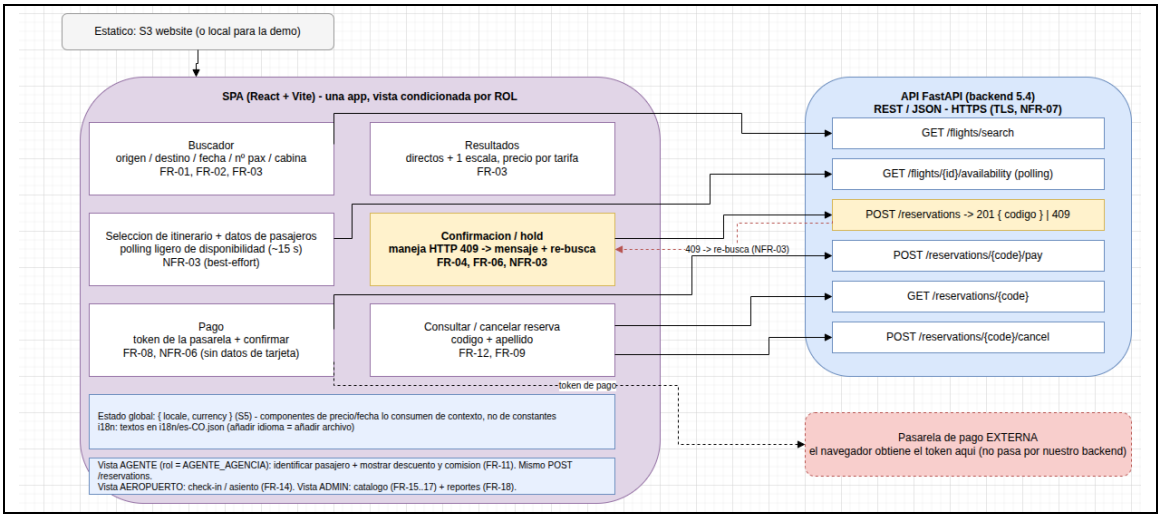
(FR-18) que consultan la **BD analítica**, no la transaccional → el análisis no compite por los locks del OLTP (NFR-08).

- El OLTP en 3FN (decisión de 5.3) es una **buena fuente** para el ETL: los datos están normalizados y sin duplicación; el ETL los transforma al esquema estrella.

4.7 Componentes de infraestructura (resumen; detalle y costos en 5.7)

Componente	Rol	Requisito	Learner Lab
API FastAPI	Monolito modular	Todos los FR	EC2 `t3.micro` (o local contra RDS para la demo)
RDS PostgreSQL (OLTP)	BD transaccional, modelo 5.3	NFR-03, NFR-04, NFR-07	`db.t3.micro`, `StorageEncrypted=true`, single-AZ (trade-off NFR-05 documentado)
RDS PostgreSQL (OLAP)	BD analítica estrella	FR-18	`db.t3.micro`
AWS Glue Job	ETL OLTP → OLAP	FR-18, 5.7	on-demand / programado
AWS Glue Data Catalog + crawlers	Metadatos de ambas BD	5.7 (requisito explícito)	crawlers JDBC
`LabRole` / perfil de instancia	Permisos	NFR-07 (sin llaves largas)	rol preexistente del Learner Lab

5 Arquitectura del frontend



5.1 SPA vs. renderizado en servidor

Decisión: Single-Page Application (SPA) — React + Vite, servida como estático (S3 website / o local para la demo), que consume la API REST del backend (5.4).

Justificación (trazable a NFR).

- **NFR-01 / NFR-02 (rendimiento de búsqueda: p95 < 800 ms directos, < 1.5 s con escalas). **La búsqueda de vuelos es** interactiva y exploratoria**: el pasajero cambia fechas, filtra por cabina, compara tarifas. Con SSR cada uno de esos cambios sería un round-trip completo con re-render de HTML en el servidor. Con SPA, la primera carga trae la aplicación y luego cada búsqueda es **una llamada JSON**: se aprovecha mejor el presupuesto de latencia del NFR y el servidor solo hace el trabajo de datos, no de presentación.
- **Coste de servidor.** El backend (monolito, un `t3.micro` en el Learner Lab) no gasta CPU renderizando vistas; se dedica a las operaciones de negocio y a proteger el inventario bajo lock.
- **Separación de responsabilidades / trazabilidad.** El mismo backend sirve a

cuatro tipos de interfaz (pasajero, agente, aeropuerto, admin — §0.3). Una SPA por rol contra una API común evita duplicar lógica de presentación en el servidor.

Alternativa considerada: SSR (Next.js server-rendered / plantillas Jinja). Descartada por lo anterior. Nota: para SEO de páginas de destino de rutas (marketing) se podría añadir prerender estático de esas páginas concretas; fuera del alcance del MVP.

5.2 Notificación en tiempo real de "asiento ya no disponible" (NFR-03)

Problema del enunciado: el usuario está en la pantalla de selección y, mientras decide, otro pasajero compra el último asiento.

Decisión: revalidación al confirmar + re-consulta ligera, sin WebSocket en el MVP.

10. La disponibilidad mostrada en resultados/selección es **orientativa** (viene de la búsqueda, que no toma lock — 5.4.3).
11. Al pulsar **"Reservar"**, el frontend llama `POST /reservations`. Si el backend —que sí toma `FOR UPDATE`— detecta que ya no hay asientos, responde **HTTP 409**.
12. La SPA captura el 409 y muestra un mensaje claro *"Ese vuelo se acaba de agotar en la cabina seleccionada"*, y vuelve a ejecutar la búsqueda* para refrescar opciones. El usuario nunca llega a un estado de sobreventa (NFR-03).
13. Mientras el usuario está en la pantalla de selección, la SPA hace un **polling ligero** (cada ~15 s) a un endpoint `GET /flights/{id}/availability` para actualizar el contador visible. Es *best-effort*: reduce sorpresas, no garantiza nada (la garantía es el 409 + la transacción del backend).

Por qué no WebSocket / SSE en el MVP: empujar cambios de inventario en tiempo real a todos los clientes que miran un vuelo exige mantener conexiones abiertas y publicar eventos desde `booking` en cada cambio de inventario — infraestructura (pub/sub, gestión de conexiones) difícil de justificar en el Learner Lab para el

volumen de una aerolínea regional (S9). Se documenta como evolución: si el negocio lo pide, `booking` publica `inventario_cambiado` y la SPA se suscribe por SSE.

5.3 Interfaz para agentes de agencia (FR-11, S3)

Decisión: misma SPA, vista condicionada por rol (no una aplicación separada).

- Tras autenticarse, si `rol = AGENTE_AGENCIA` la SPA habilita:
- un paso extra para **identificar al pasajero** a nombre de quien se reserva;
- la visualización del **descuento de agencia** aplicado y de la **comisión** estimada;
- acceso a "mis reservas de la agencia".
- El **flujo de reserva es el mismo** (misma llamada `POST /reservations`), lo que mantiene una sola ruta de código y una sola prueba de concurrencia. Coherente con la decisión de backend de 5.4.5.
- Una interfaz B2B dedicada (facturación consolidada, tarifas negociadas) se **diseña** junto con la API B2B pero **no** se construye en el MVP.

5.4 Multi-moneda y multi-idioma (S5)

Decisión: preparado en la estructura, una sola configuración activa en el MVP (COP, es-CO).

- **Idioma:** textos externalizados en archivos de recursos (`i18n/es-CO.json`); añadir un idioma = añadir un archivo + entrada en el selector. Ningún texto hardcodeado en componentes.
- **Moneda:** los precios llegan de la API con `{ monto, moneda }` (la columna `moneda` ya existe en `TARIFA`/`RESERVA` — 5.3). El frontend **formatea** según `Intl.NumberFormat` con el locale activo; **no convierte** importes (la conversión, si se necesita, es responsabilidad del backend con una tabla de tipos

de cambio).

- **Implicación estructural:** el estado global de la SPA incluye `locale` y `currency`; los componentes de precio y fecha los consumen de contexto, no de constantes. Esto es lo único que el MVP "paga" por adelantado para no rehacer la UI al internacionalizar.

5.5 Componentes de la SPA (pasajero) y su mapeo a la API

Pantalla / componente	Endpoint(s) del backend	FR
Buscador (origen, destino, fecha, nº pax, cabina)	`GET /flights/search`	FR-01, FR-02, FR-03
Resultados (directos + con escala, precio por tarifa)	— (render del resultado)	FR-03
Selección de itinerario + datos de pasajeros	`GET /flights/{id}/availability` (polling)	NFR-03
Confirmación / hold	`POST /reservations` → código, o **409**	FR-04, FR-06, NFR-03
Pago	(token del proveedor) + `POST /reservations/{code}/pay`	FR-08, NFR-06
Consultar reserva (código + apellido)	`GET /reservations/{code}`	FR-12
Cancelar	`POST /reservations/{code}/cancel`	FR-09

5.6 Ajustes durante la implementación

El frontend se implementó en `frontend/` (React 18 + Vite 5 + react-router 6).

Corresponde con el diseño de arriba; las diferencias y decisiones concretas:

#	Diseño (5.5.x)	Implementación	Motivo
F1	SPA React + Vite servida como estático	Igual. `npm run build` → `dist/` estático; en desarrollo, proxy `/api` → `localhost:8000`	—
F2	Pantallas búsqueda → selección → confirmación → código, y consultar reserva	Implementadas las 4 (`BuscarPage`, `SeleccionPage`, `ConfirmacionPage`, `MiReservaPage`)	alcance MVP de 5.5.5
F3	Pago y cancelación como pantallas de apoyo	**No** se construyeron (el backend tampoco expone `POST /pay` ni `/cancel` en el MVP — ver `docs/06` D6). La UI las omite en vez de mostrar	coherencia con el alcance del backend

		botones muertos	
F4	Manejo del 409 al confirmar (5.5.2)	`api.js` lanza `ApiError` con `status`; `SeleccionPage` captura 409 → mensaje + `navigate("/buscar")`	—
F5	Polling ligero de disponibilidad cada ~15 s (5.5.2)	`SeleccionPage` hace `setInterval` de 15 s a `GET /flights/{id}/availability` por cada tramo; se limpia al desmontar	—
F6	i18n: textos externalizados, sin literales en componentes (5.5.4)	`src/i18n/es-CO.json` + `makeT(locale)`; todos los textos visibles pasan por `t()`	—
F7	Multi-moneda: formatear con `Intl`, no convertir (5.5.4)	`useFormatters().money(monto, moneda)` usa `Intl.NumberFormat` con el `moneda` que devuelve la API	—
F8	Estado global `locale`/`currency` por contexto (5.5.4)	`ConfigProvider` / `useConfig`; en el MVP los valores son fijos (`es-CO`, `COP`), pero los componentes los leen de contexto	—
F9	Vista por rol para agentes de agencia (5.5.3)	El flujo es el mismo; `SeleccionPage` permite elegir canal `AGENCIA` + `id_agencia`, y `MiReservaPage` muestra la comisión cuando `monto_comision > 0`. No hay login ni rol persistente (el backend del MVP no autentica)	el MVP no tiene autenticación; se expone la capacidad de canal sin la gestión de sesión
F10	CORS	Se añadió `CORSMiddleware` al backend (`app/main.py`), configurable con `CORS_ORIGINS` en `.env`	la SPA es un origen distinto (Vite :5173 o el sitio S3)

Ninguna diferencia obligó a cambiar el modelo ER ni la API; F3/F9 son

consecuencia del alcance ya acordado del backend, no cambios de diseño.

6 Primera implementación del backend

Código: `backend/` · Guía de ejecución: `backend/README.md`

Referencias: modelo en 03-modelo-entidad-relacion.md, arquitectura en 04-arquitectura-backend.md, NFR en 02-requisitos-no-funcionales.md.

6.1 Alcance implementado

Operación	Endpoint	FR	Estado
Buscar vuelos disponibles (directos + 1	`GET /flights/search`	FR-01, FR-02, FR-03	✅

escala)			
Crear reserva (multi-tramo, hold) con control de concurrencia	`POST /reservations`	FR-04, FR-05, FR-06	✅
Consultar reserva existente (con historial)	`GET /reservations/{codigo}`	FR-12, FR-13	✅
Disponibilidad puntual (polling del frontend)	`GET /flights/{id}/availability`	NFR-03 (best-effort)	✅
Salud del servicio	`GET /health`	NFR-05	✅

Stack: FastAPI + SQLAlchemy 2.0 + Alembic sobre **PostgreSQL 16**. Un solo servicio (monolito modular, 5.4.1). Módulos implementados: `search`, `booking` (dueño de la concurrencia), `db`, `config`, `models`, `schemas`.

Base de datos: implementa el modelo ER de 5.3 con **19 tablas**, en **3FN**, con los `CHECK`, `UNIQUE` y FKs del diccionario de datos. Nivel de normalización y motor son los definidos y justificados por el equipo en 5.3.

6.2 Mecanismo de concurrencia y cómo se probó

Mecanismo (fiel a 5.4.2): bloqueo pesimista — `SELECT ... FOR UPDATE` sobre las filas de `inventario_asientos` de cada segmento, dentro de la transacción de reserva, con orden determinista por `id_instancia_vuelo` (previene deadlock).

Código: `backend/app/services/booking.py`, función `crear_reserva`.

Barreras (defensa en profundidad):

14. Validación en `booking`: `disponibles < num_pax` → **HTTP 409** (NFR-03).
15. `CHECK (asientos_vendidos + asientos_retenidos <= asientos_totales)` a nivel de motor (migración `0001`) — última barrera si algo se escapa de la lógica.
16. Trigger `evento_reserva_append_only` — rechaza `UPDATE`/`DELETE` sobre la bitácora de auditoría (NFR-09).

Prueba automatizada 1 — capa de servicio, N hilos concurrentes

`backend/tests/test_concurrencia.py`.

20 hilos, **cada uno con su propia conexión/transacción a PostgreSQL** (como N procesos web reales), intentan reservar el último asiento a la vez.

Evidencia — `docs/evidencia/prueba-concurrencia.txt`:

```
============================================================
PRUEBA DE CONCURRENCIA  -  20 clientes por 1 asiento
============================================================
  reservas OK (hold)     : 1   ['OK:MXMVZ3']
  rechazos HTTP 409      : 19
  otros errores          : 0   []
  inventario final       : vendidos=59 retenidos=1 totales=60
  invariante (v+r <= t)  : 60 <= 60  -> OK
============================================================
2 passed
```

Prueba automatizada 2 — API HTTP en vivo

`backend/tests/concurrencia_http.py`.

20 `POST /reservations` HTTP **simultáneos** contra el servidor uvicorn.

Evidencia — `docs/evidencia/prueba-concurrencia-http.txt`:

```
============================================================
POST /reservations x20 simultaneos por 1 asiento
  201 Created : 1   ['201 CCXZRV']
  409 Conflict: 19
  otros       : 0   []
============================================================
OK: exactamente 1 reserva, sin sobreventa
```

Conclusión: en ambas pruebas se cumple el NFR-03 — exactamente **1** reserva se lleva el asiento, las otras 19 reciben **409**, y el invariante `vendidos + retenidos ≤ totales` se mantiene: **cero sobreventa**.

6.3 Decisiones de implementación y desviaciones frente a 3 / 4

#	Punto	¿Desviación?	Detalle y justificación
D1	**`SELECT ... FOR UPDATE` sin filtrar por `cabina` en el lock**	Ajuste menor	En `booking.crear_reserva` el `FOR UPDATE` bloquea **todas** las filas de inventario de las instancias implicadas (todas las cabinas), no solo la cabina pedida. Motivo: simplicidad y correctitud (bloquea de más, nunca de menos); el número de cabinas por vuelo es 2. Si en el futuro hubiera muchas cabinas se añadiría `AND cabina = :c` al `WHERE ... FOR UPDATE`. No afecta a 5.3 ni a la garantía de

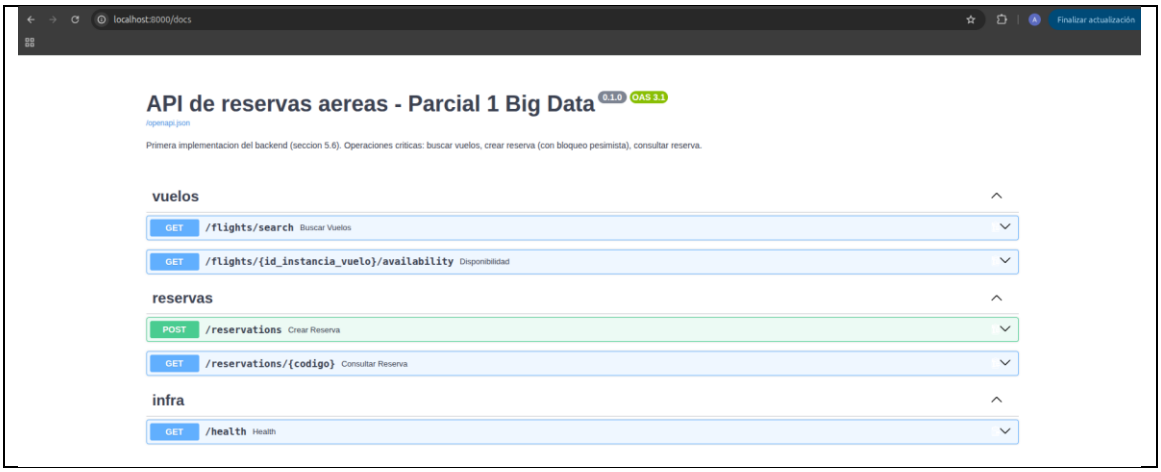
			NFR-03.
D2	**Precio del segmento = `tarifa.monto_precio * num_pasajeros`**	Precisión del diseño	5.3 define `segmento_itinerario.monto_segmento` sin especificar si es por pasajero o total. Se implementó como **total del tramo** (precio × nº pax). `reserva.monto_total` = suma de segmentos, con descuento de agencia aplicado. Se documenta aquí; no cambia el modelo.
D3	**Estados en español**	Coherencia	El código usa `RETENIDA / CONFIRMADA / CANCELADA / EXPIRADA` (5.3) y cabinas `ECONOMICA / EJECUTIVA`. Sin cambio de diseño.
D4	**`asientos_totales` como columna, no derivada de `config_cabina`**	Ya previsto en 5.3	Confirmado en implementación: `seed` copia `num_asientos` de `config_cabina` a `inventario_asientos.asientos_totales` al crear la instancia. Un cambio futuro de aeronave no altera vuelos ya vendidos (decisión de 5.3.1).
D5	**Búsqueda con escala limitada a 1 conexión y ventana `[45, 360]` min**	Ya previsto en 5.4.3	Parámetros en `config.py` (`MIN/MAX_CONNECTION_MINUTES`). Implementado tal cual el diseño.
D6	**`pago`, `cancelar`, `checkin`, `jobs` no implementados en el MVP**	Alcance declarado	5.6 exige buscar / crear / consultar. El resto está **diseñado** (5.4) y modelado (5.3, tablas `pago`, `billete`, `asignacion_asiento` ya creadas) pero fuera del alcance de esta entrega. El hold **no expira automáticamente** todavía (falta el job `FR-07`); `retenida_hasta` se calcula y guarda.

Ningún cambio obligó a modificar el modelo ER de 3 — todas las tablas y restricciones del diccionario de datos se implementaron sin alteración. La sección 5.3.7 queda sin entradas de corrección.

6.4 Entregables de esta subsección

- **Código fuente:** `backend/` (se subirá a GitHub).
- **Esquema de BD:** migración Alembic `0001_esquema_inicial.py` (`alembic upgrade head` lo crea).
- **Guía de ejecución:** `backend/README.md`.
- **Evidencia de la prueba de concurrencia:** `docs/evidencia/` (salidas de ambas pruebas) + los propios tests reproducibles.

https://github.com/asantiagodiaz/BD_parcial1.git



Evidencia — prueba de concurrencia (capa de servicio, 20 hilos simultaneos)

```
============================= test session starts ==============================
platform linux -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 --
/home/santiago_diaz/Universidad/Noveno/Big_data/Parcial1/backend/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/santiago_diaz/Universidad/Noveno/Big_data/Parcial1/backend
configfile: pyproject.toml
plugins: anyio-4.15.0
collecting ... collected 2 items

tests/test_concurrencia.py::test_no_sobreventa_con_N_clientes_simultaneos
============================================================
PRUEBA DE CONCURRENCIA  -  20 clientes por 1 asiento
============================================================
  reservas OK (hold)     : 1   ['OK:MXMVZ3']
  rechazos HTTP 409      : 19
  otros errores          : 0   []
  inventario final       : vendidos=59 retenidos=1 totales=60
  invariante (v+r <= t)  : 60 <= 60  -> OK
============================================================
PASSED
tests/test_concurrencia.py::test_segunda_tanda_ya_no_hay_cupo PASSED

============================== 2 passed in 0.74s ===============================
```

Evidencia — prueba de concurrencia (API HTTP en vivo, 20 peticiones)

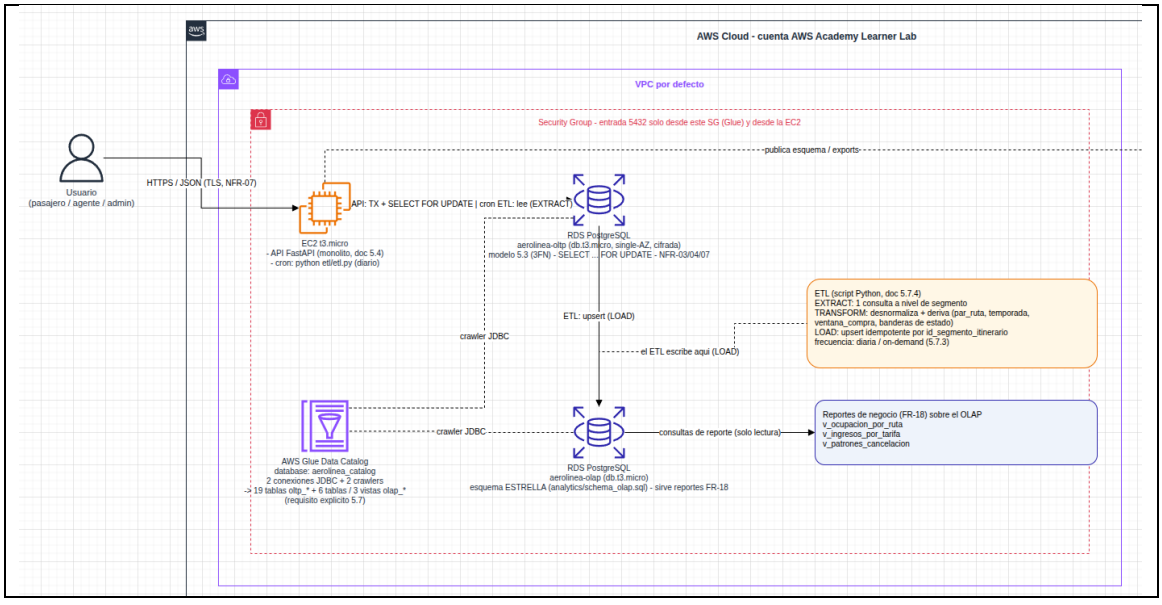
```
============================================================
POST /reservations x20 simultaneos por 1 asiento
  201 Created : 1   ['201 CCXZRV']
  409 Conflict: 19
  otros       : 0   []
============================================================
OK: exactamente 1 reserva, sin sobreventa
```

Evidencia — prueba de concurrencia contra la API desplegada en AWS

```
# Prueba de concurrencia contra la API desplegada en AWS (Learner Lab)
# EC2 t3.small -> RDS PostgreSQL aerolinea-oltp. 2026-09-03T02:55:28Z
# 20 POST /reservations simultaneos por el ultimo asiento; se espera 1x201 + 19x409.

instancia=1  tarifa=1  ECONOMICA disponibles=1
============================================================
POST /reservations x20 simultaneos por 1 asiento
  201 Created : 1   ['201 JZA83C']
  409 Conflict: 19
  otros       : 0   []
============================================================
OK: exactamente 1 reserva, sin sobreventa
```

7 Implementación AWS — ETL, Glue Data Catalog y costos



7.1 Qué se implementó

Requisito del enunciado	Implementación
BD transaccional con el modelo de 5.3	RDS PostgreSQL `aerolinea-oltp` (esquema de la migración `0001`)
2ª BD PostgreSQL analítica (OLAP) que recibe copia de ≥1 tabla/vista vía ETL	RDS PostgreSQL `aerolinea-olap`, esquema **estrella** (`schema_olap.sql`)
Proceso ETL entre ambas	Script Python `etl/etl.py` — `EXTRACT` (1 consulta que une el OLTP a nivel de segmento) → `TRANSFORM` (desnormaliza + deriva columnas) → `LOAD` (upsert idempotente)
Ambas BD registradas en AWS Glue Data Catalog	El stack CDK (`infra/cdk/lib/aerolinea-stack.ts`) crea 1 database `aerolinea_catalog`, 2 conexiones JDBC y 2 crawlers (prefijos `oltp_` / `olap_`)

Evidencia local (mismo código, PostgreSQL local): el ETL cargó 14 segmentos de reserva y las 3 vistas de negocio responden — ver `docs/evidencia/etl-vistas-negocio.txt`.

7.2 ¿Qué se copia al OLAP y cómo?

Qué: las **reservas a nivel de segmento** (`fact_reserva_segmento`, grano = un tramo de una reserva) más sus dimensiones `dim_ruta`, `dim_vuelo`, `dim_tarifa`, `dim_tiempo`, `dim_canal`.

No se copia tal cual — se transforma:

- **Desnormalización:** el hecho ya trae `id_ruta`, `id_tarifa`, `id_canal`, etc.; una consulta analítica no necesita recorrer 8 tablas como en el OLTP.
- **Columnas derivadas:** `par_ruta` (`BOG-MDE`), `ventana_compra` (`anticipada`/`última hora` según `dias_anticipacion_min`), `temporada` (`ALTA`/`BAJA` según el mes), banderas `es_confirmada`/`es_cancelada`/`es_expirada`.
- **Comisión prorrateada:** `reserva.monto_comision` (total) se reparte por segmento según su peso en el `monto_total`.
- **`dim_tiempo`** se genera en el ETL (no existe en el OLTP): un registro por fecha con año/mes/día de semana/temporada.

Por qué el esquema OLAP ≠ OLTP (relación con la normalización de 5.3): el OLTP está en **3FN** porque su prioridad es la **integridad en escritura** (evitar anomalías, `SELECT ... FOR UPDATE` sobre filas pequeñas — NFR-03/04). El OLAP tiene la prioridad opuesta: **lecturas analíticas rápidas** con pocos joins → esquema estrella desnormalizado. Son dos modelos para dos cargas de trabajo distintas; por eso también viven en **instancias separadas** (el análisis no compite por los locks del transaccional — NFR-08).

Las 3 preguntas de negocio del enunciado se responden con vistas sobre el hecho:

Pregunta	Vista
Ocupación por ruta	`v_ocupacion_por_ruta` (pax confirmados / asientos ofertados, por ruta y mes)
Ingresos por tarifa	`v_ingresos_por_tarifa` (ingreso y comisión por clase de tarifa, cabina, ventana de compra)
Patrones de cancelación	`v_patrones_cancelacion` (% cancelación e ingreso perdido por ruta, tarifa, canal, temporada)

7.3 ¿Con qué frecuencia corre el ETL?

Decisión: diaria / on-demand (cron), no event-triggered.

Opción	Freshness	Costo / complejidad	¿Se elige?
Una sola vez	datos congelados	mínimo	No — la gerencia quiere seguimiento continuo
Periódica (diaria)	≤ 24 h	1 ejecución/día, trivial	**Sí.** El análisis comercial (ocupación, ingresos, cancelaciones) se revisa por día/semana; no necesita datos al segundo.
Event-triggered (cada reserva dispara ETL)	segundos	alto: cola/lambda por evento, muchas ejecuciones, carga constante sobre el OLTP	No — sobredimensionado para el volumen de una aerolínea regional (S9)

El script es idempotente (`upsert` por `id_segmento_itinerario`), así que una ejecución fallida se puede reintentar sin duplicar. En AWS: un cron en la EC2 (`0 3 * * * python etl/etl.py`) o un EventBridge Schedule.

7.4 ¿Por qué el ETL es un script Python y no un Glue Job?

	Script Python (elegido)	AWS Glue Job (PySpark)
Complejidad de configuración en el Learner Lab	baja: solo necesita las 2 URLs de conexión	alta: rol de Glue, conexión JDBC, driver, VPC, endpoints, script en S3
Legibilidad / defensa oral	alta: 1 archivo, SQL explícito	media: DynamicFrames, transformaciones Spark
Escala (volumen de una aerolínea regional, S9)	suficiente (miles de filas/día)	pensado para TB; ocioso aquí
Costo	corre en la EC2 que ya existe → **$0 extra**	DPU-hora facturado por ejecución
Portabilidad	corre igual en local y en AWS	atado a Glue

Glue Job PySpark queda documentado como evolución: si el volumen creciera a millones de reservas/día, el `EXTRACT`/`TRANSFORM` se movería a un Glue Job que lee el OLTP por JDBC en paralelo y escribe el OLAP; el `schema_olap.sql` no cambiaría.

Glue sí se usa — para el **Data Catalog**, que es requisito explícito.

7.5 Conexión de ambas BD al Glue Data Catalog

El stack CDK (`infra/cdk/lib/aerolinea-stack.ts`) declara los recursos de Glue como parte de la infraestructura:

17. **Database del catálogo:** `aerolinea_catalog` (`CfnDatabase`).
18. **2 conexiones JDBC** (`conn-oltp`, `conn-olap`): URL `jdbc:postgresql://<endpoint-rds>:5432/aerolinea`, usuario/clave, y `PhysicalConnectionRequirements` (subred + security group de la VPC por defecto del Lab, con el puerto 5432 abierto).
19. **2 crawlers** (`crawler-oltp`, `crawler-olap`) con el ARN de `LabRole`, target `JdbcTargets` apuntando a `aerolinea/public/%`, y prefijos de tabla

`oltp_` / `olap_` para distinguirlas en el catálogo.

20. Tras el despliegue se arrancan los crawlers (`aws glue start-crawler`) y en unos minutos las 19 tablas del OLTP y las 6 tablas + 3 vistas del OLAP quedan visibles como metadatos:

`aws glue get-tables --database-name aerolinea_catalog`.

Acceso de red: las RDS son `publicly-accessible` en el MVP (Learner Lab) con un security group que permite 5432 desde el propio SG (Glue) y desde la EC2. El `LabRole` ya tiene los permisos de Glue y de lectura que necesitan los crawlers.

7.6 Justificación de los servicios AWS

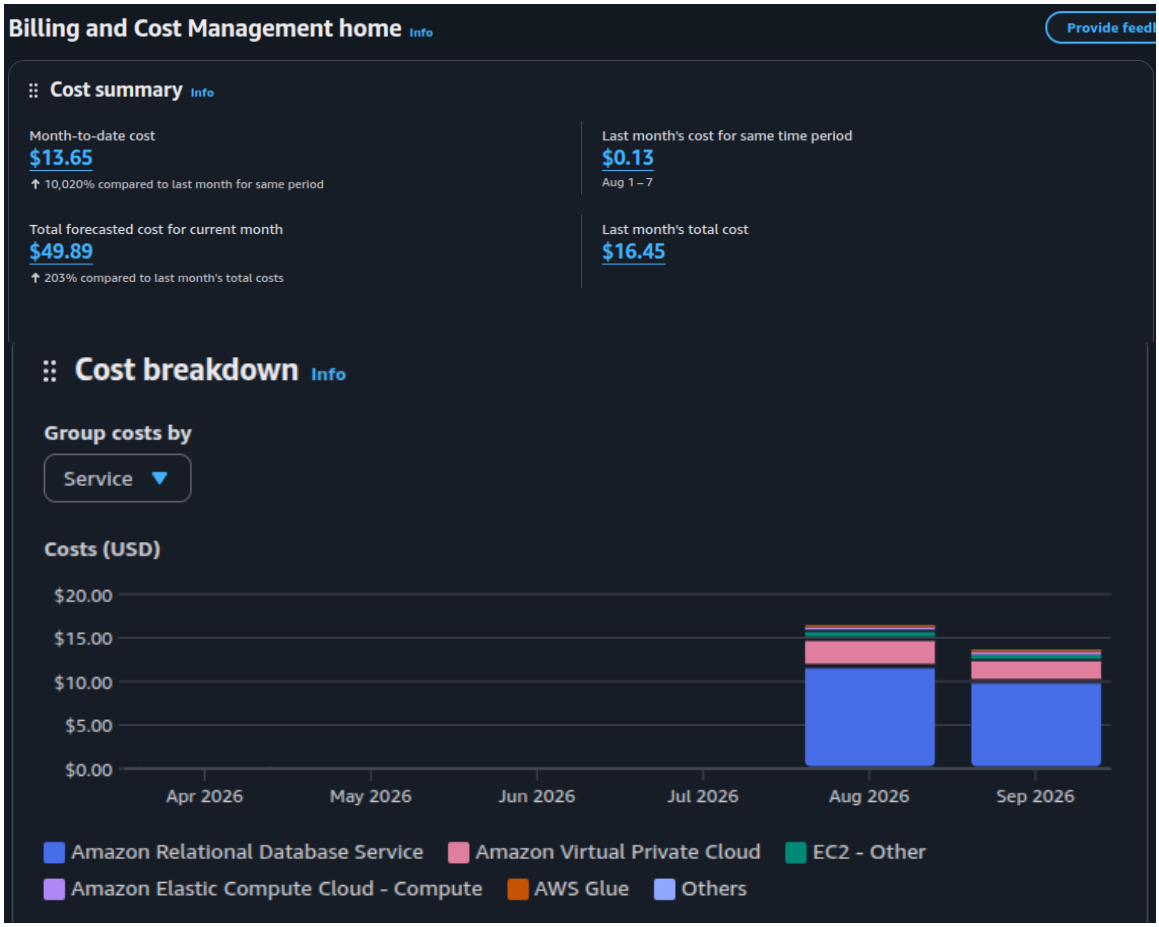
Cada servicio se conecta a (1) un FR/NFR concreto de 5.1/5.2 y (2) uno o más pilares del **AWS Well-Architected Framework** (Seguridad, Fiabilidad, Eficiencia del rendimiento, Optimización de costos, Excelencia operativa, Sostenibilidad).

AWS Service	Alternativa considerada	Requisito (FR/NFR) que soporta la elección	Pilar(es) Well-Architected
RDS PostgreSQL — `aerolinea-oltp` (`db.t3.micro`, single-AZ, cifrada)	Aurora Serverless v2; PostgreSQL autogestionado en EC2	**NFR-03/04**: `SELECT ... FOR UPDATE` y transacciones ACID exigen un motor relacional real. **NFR-07**: cifrado en reposo (`StorageEncrypted`) gestionado.	**Fiabilidad** (backups automáticos, recuperación gestionada). **Seguridad** (cifrado KMS, sin gestionar parches del SO). **Optimización de costos** (t3.micro cabe en el presupuesto del Lab; Aurora es más caro para este volumen).
RDS PostgreSQL — `aerolinea-olap` (`db.t3.micro`)	Redshift; Athena sobre S3 (Parquet)	Requisito explícito 5.7 (*"segunda base PostgreSQL analítica"*). **FR-18**: servir reportes de ocupación/ingresos/cancelaciones.	**Eficiencia del rendimiento** (esquema estrella = consultas analíticas con pocos joins).

			Optimización de costos (Redshift mínimo cuesta ~10× más; innecesario para el volumen de S9).
Script ETL en la EC2 (Python + cron)	AWS Glue Job (PySpark); AWS Data Pipeline; DMS	**FR-18** + relación OLTP→OLAP del pipeline (5.4.6). El volumen (S9) no justifica Spark.	**Optimización de costos** (reutiliza la EC2 de la API → $0 extra; Glue factura por DPU-hora). **Excelencia operativa** (un archivo legible, idempotente, versionado). **Sostenibilidad** (sin clúster ocioso).
AWS Glue Data Catalog + 2 crawlers JDBC	Registro manual de tablas; catálogo propio	Requisito explícito 5.7 (ambas BD *"visibles y consultables como metadatos desde Glue"*).	**Excelencia operativa** (metadatos centralizados y auto-descubiertos; base para futuros Athena/Glue Jobs).
EC2 `t3.small` (API FastAPI + cron ETL)	AWS Lambda + API Gateway; Fargate; Elastic Beanstalk	Todos los FR (host del monolito de 5.4). **NFR-01/02**: latencia predecible sin cold start.	**Optimización de costos** (una instancia pequeña sirve API + ETL; Lambda añadiría complejidad de empaquetado y VPC). **Excelencia operativa** (despliegue simple: `git pull` + `systemd`).
IAM — `LabRole` / perfil de instancia	Usuario IAM con access keys de larga duración	**NFR-07**: no incrustar credenciales; el Learner Lab rota la sesión.	**Seguridad** (credenciales temporales,

			principio de mínimo privilegio del Lab).
VPC por defecto + Security Group (5432 solo desde el SG y la EC2)	RDS público sin restricción; VPC dedicada con subredes privadas + NAT	**NFR-07**: aislar la BD del acceso público general.	**Seguridad** (superficie de red mínima). *Trade-off documentado:* subredes privadas + NAT serían lo ideal en producción; el Lab no lo permite cómodamente, se deja RDS accesible con SG restringido.
Amazon S3 (respaldo de `schema_olap.sql`, exports de reportes)	EBS; almacenamiento local	Soporte a **FR-18** (compartir extractos de reportes).	**Optimización de costos** (centavos/mes). **Fiabilidad** (durabilidad 11 nueves).

7.7 Proyección de costos mensual



Supuestos explícitos:

- Región **`us-east-1`**, precios públicos on-demand (sept. 2026), USD, sin capa

gratuita (la cuenta del Lab no la tiene).

- Se dan **dos columnas**: **730 h/mes (24/7)**, que es lo que calcula la AWS

Pricing Calculator por defecto; y **uso real del Lab** (~6 h/día × 20 días =

120 h/mes, apagando con `deploy.sh destroy` al terminar cada sesión).

- Volumen actual (S9): ~20 rutas, ~600 instancias de vuelo/mes, ~5 000 reservas/mes

→ datos del orden de **decenas de MB**.

- ETL: **1 ejecución/día**, ~30 s, sobre la EC2 ya contabilizada.

Desglose por servicio (formato AWS Pricing Calculator)

Servicio	Configuración	Precio unitario	Cómputo 24/7	Uso real Lab

				(120 h)
Amazon RDS — `aerolinea-oltp`	PostgreSQL 16, `db.t3.micro` (2 vCPU burst, 1 GiB), Single-AZ	$0.018 / h	$13.14	$2.16
Amazon RDS — `aerolinea-olap`	PostgreSQL 16, `db.t3.micro`, Single-AZ	$0.018 / h	$13.14	$2.16
RDS — almacenamiento	gp3, 20 GB × 2 instancias = 40 GB	$0.115 / GB-mes	$4.60	$4.60
RDS — backups	Retención 1 día; volumen ≤ tamaño de la BD → dentro del backup gratuito	—	$0.00	$0.00
Amazon EC2 — `aerolinea-api`	`t3.small` (2 vCPU burst, 2 GiB), Linux on-demand	$0.0208 / h	$15.18	$2.50
EC2 — almacenamiento	EBS gp3, 20 GB, cifrado	$0.08 / GB-mes	$1.60	$1.60
AWS Glue — Data Catalog	~28 tablas (19 `oltp_*` + 9 `olap_*`). Primer 1 M de objetos y de solicitudes: gratis	$0.00	$0.00	$0.00
AWS Glue — Crawlers	2 crawlers × ~2 min/ejecución. Mín. facturable 10 min = 1/6 DPU-h. 24/7: 2/día; Lab: 1/sesión	$0.44 / DPU-h	$4.40 (≈60 ejec.)	$0.15 (≈20 ejec.)
Amazon S3	Tarball de código (~0.2 MB) + `schema_olap.sql`; pocos GET/PUT	$0.023 / GB-mes	$0.02	$0.02
Transferencia de datos	EC2 ↔ RDS ↔ Glue dentro de la misma VPC/AZ; salida < 1 GB/mes (gratis)	—	$0.00	$0.00
		TOTAL	**≈ $52.2 / mes**	**≈ $15.9 / mes**

El uso real en el Lab (~$16/mes) cabe holgadamente en el crédito asignado

(~$50–100). El total 24/7 (~$52/mes) es la cifra que devuelve la AWS Pricing

Calculator para este conjunto de recursos.

Escenario ×10 (volumen o frecuencia multiplicados por 10)

Interpretación: **10× reservas** (~50 000/mes, datos de cientos de MB a ~2 GB) y/o

ETL cada 2–3 h en vez de diario. Cifras sobre la columna **24/7**.

Servicio	Cambio con ×10	Costo base 24/7	Costo ×10 24/7	Comentario / pilar
RDS OLTP	`db.t3.micro` → **`db.t3.small`** (2 GiB) para absorber más escrituras y el lock bajo carga	$13.14	~$26.30	El cuello de botella sigue siendo la contención del lock (NFR-04), no el tamaño; se sube **una talla**, no ×10. **Optimización de costos:** escalar vertical 1 paso antes que sharding.
RDS OLAP	`db.t3.micro` → **`db.t3.small`**	$13.14	~$26.30	El hecho crece linealmente; las vistas agregan. **Eficiencia del rendimiento.**
RDS almacenamiento	40 GB → ~120 GB	$4.60	~$13.80	Lineal con el volumen.
EC2 (API + ETL)	ETL ×8 más frecuente, pero cada corrida sigue siendo segundos; la API aguanta	$15.18 + $1.60	~$16.78 (sin cambio)	El script Python escala bien aquí. **Sostenibilidad:** sin infra nueva.
Glue Crawlers	ejecutar tras cada carga (×8/día) → ~40 DPU-h/mes	$4.40	~$17.60	Se reduce corriendo el crawler **solo cuando cambia el esquema**, no en cada carga → volvería a ~$4. **Optimización de costos.**
Glue Data Catalog + S3	~280 objetos, aún gratis; S3 ~2 GB	$0.02	~$0.05	Dentro del nivel gratuito del catálogo.
TOTAL ×10 (24/7)		**≈ $52 / mes**	**≈ $117 / mes**	**~2.25×** el costo base, **no ×10**

Conexión con los pilares Optimización de costos y Sostenibilidad: al

multiplicar el volumen por 10, el costo sube solo ~2,25× porque (a) los servicios

de datos escalan **verticalmente un paso**, no proporcionalmente; (b) el ETL y el catálogo **no escalan con el volumen** en este rango (script sobre la EC2 existente, catálogo gratuito); (c) el crawler se puede desacoplar de la carga para no pagar cómputo redundante. Si el volumen siguiera creciendo (×100), la migración recomendada sería OLAP → **Athena sobre S3/Parquet** (se paga por consulta y por almacenamiento comprimido, cero cómputo ocioso) y ETL → **Glue Job**.

Estimado en la AWS Pricing Calculator

El estimado con estos mismos recursos y precios está guardado en `https://calculator.aws/#/estimate` — [pegar aquí el enlace público del estimado y una captura de la pantalla de resumen].

7.8 Diagrama de arquitectura AWS

`diagrams/07-arquitectura-aws.drawio` — muestra: navegador → EC2 (FastAPI + cron ETL) → RDS OLTP; EC2/ETL → RDS OLAP; crawlers JDBC desde Glue a ambas RDS → Glue Data Catalog; todo en la VPC por defecto con el security group restringido; `LabRole` como identidad.

Evidencia — ETL + 3 vistas de negocio + idempotencia (local)

```
============================================================
 EVIDENCIA ETL  OLTP -> OLAP  +  vistas de negocio (5.7)
 (ejecutado localmente; mismo codigo que en AWS)
============================================================

--- ejecucion del ETL ---
ETL OK:
  segmentos_procesados  : 14
  confirmados           : 9
  cancelados            : 3
  modo                  : full
  ejecutado             : 2026-09-02T23:06:45+00:00

--- tablas en el OLAP ---
                List of relations
 Schema |         Name          | Type  |   Owner
--------+-----------------------+-------+-----------
 public | dim_canal             | table | aerolinea
```

```
 public | dim_ruta              | table | aerolinea
 public | dim_tarifa            | table | aerolinea
 public | dim_tiempo            | table | aerolinea
 public | dim_vuelo             | table | aerolinea
 public | fact_reserva_segmento | table | aerolinea
(6 rows)



=== PREGUNTA 1: OCUPACION POR RUTA ===
 par_ruta | mes | temporada | vuelos | pasajeros_confirmados | asientos_ofertados |
pct_ocupacion
----------+-----+-----------+--------+-----------------------+--------------------+---------
------
 BOG-CLO  |   9 | BAJA      |      3 |                     0 |                318 |
0.0
 BOG-CTG  |   9 | BAJA      |      3 |                     2 |                486 |
0.4
 BOG-MDE  |   9 | BAJA      |      6 |                     6 |                408 |
1.5
 MDE-CTG  |   9 | BAJA      |      2 |                     1 |                136 |
0.7
(4 rows)

=== PREGUNTA 2: INGRESOS POR TARIFA ===
 clase_tarifa |  cabina   | ventana_compra | mes | ingreso_confirmado | comision_pagada |
pasajeros | ingreso_promedio_pax
--------------+-----------+----------------+-----+--------------------+-----------------+---
--------+----------------------
 PROMO        | ECONOMICA | anticipada     |   9 |         1940000.00 |        32760.00 |
9 |            215555.56
(1 row)

=== PREGUNTA 3: PATRONES DE CANCELACION ===
 par_ruta | clase_tarifa |  canal  | temporada | segmentos_totales | segmentos_cancelados |
pct_cancelacion | ingreso_perdido_cancelaciones
----------+--------------+---------+-----------+-------------------+----------------------+-
----------------+-------------------------------
 BOG-CLO  | PROMO        | AGENCIA | BAJA      |                 2 |                    1 |
50.0 |                     204000.00
 BOG-CLO  | PROMO        | DIRECTO | BAJA      |                 1 |                    1 |
100.0 |                     204000.00
 BOG-CTG  | PROMO        | AGENCIA | BAJA      |                 1 |                    0 |
0.0 |
 BOG-CTG  | PROMO        | DIRECTO | BAJA      |                 2 |                    0 |
0.0 |
 BOG-MDE  | PROMO        | AGENCIA | BAJA      |                 1 |                    0 |
0.0 |
 BOG-MDE  | PROMO        | DIRECTO | BAJA      |                 5 |                    0 |
0.0 |
 MDE-CTG  | PROMO        | DIRECTO | BAJA      |                 2 |                    1 |
50.0 |                     236000.00
(7 rows)
```

```
--- idempotencia: 2a ejecucion (upsert) no duplica ---
  segmentos_procesados  : 14
 filas en fact_reserva_segmento: 14
```

Evidencia — Glue Data Catalog: crawlers y tablas catalogadas (AWS)

```
# Glue Data Catalog - tablas catalogadas por los crawlers (Learner Lab)
# database: aerolinea_catalog   2026-09-03T02:55:39Z
# crawler-oltp (prefijo oltp_) sobre RDS aerolinea-oltp ; crawler-olap (prefijo olap_) sobre
RDS aerolinea-olap

## Estado de los crawlers
-----------------------------------------
|              GetCrawler                |
+--------+----------------+---------------+
| Estado |    Nombre      |  UltimoCrawl  |
+--------+----------------+---------------+
|  READY |  crawler-oltp  |  SUCCEEDED    |
+--------+----------------+---------------+
-----------------------------------------
|              GetCrawler                |
+--------+----------------+---------------+
| Estado |    Nombre      |  UltimoCrawl  |
+--------+----------------+---------------+
|  READY |  crawler-olap  |  SUCCEEDED    |
+--------+----------------+---------------+

## Tablas en el Data Catalog
-------------------------------------------------
|                  GetTables                    |
+-----------------------------------------------+
|  olap_aerolinea_public_dim_canal              |
|  olap_aerolinea_public_dim_ruta               |
|  olap_aerolinea_public_dim_tarifa             |
|  olap_aerolinea_public_dim_tiempo             |
|  olap_aerolinea_public_dim_vuelo              |
|  olap_aerolinea_public_fact_reserva_segmento  |
|  olap_aerolinea_public_v_ingresos_por_tarifa  |
|  olap_aerolinea_public_v_ocupacion_por_ruta   |
|  olap_aerolinea_public_v_patrones_cancelacion |
|  oltp_aerolinea_public_aeronave               |
|  oltp_aerolinea_public_aeropuerto             |
|  oltp_aerolinea_public_agencia                |
|  oltp_aerolinea_public_alembic_version        |
|  oltp_aerolinea_public_asignacion_asiento     |
|  oltp_aerolinea_public_billete                |
|  oltp_aerolinea_public_config_cabina          |
|  oltp_aerolinea_public_cuenta_usuario         |
|  oltp_aerolinea_public_evento_reserva         |
|  oltp_aerolinea_public_instancia_vuelo        |
|  oltp_aerolinea_public_inventario_asientos    |
|  oltp_aerolinea_public_itinerario             |
|  oltp_aerolinea_public_pago                   |
|  oltp_aerolinea_public_pasajero               |
```

```
|  oltp_aerolinea_public_reserva                |
|  oltp_aerolinea_public_reserva_pasajero       |
|  oltp_aerolinea_public_ruta                   |
|  oltp_aerolinea_public_segmento_itinerario    |
|  oltp_aerolinea_public_tarifa                 |
|  oltp_aerolinea_public_vuelo_programado       |
+-----------------------------------------------+
```

Evidencia — 3 vistas de negocio ejecutadas contra la RDS OLAP (AWS)

```
# 3 vistas de negocio ejecutadas contra la RDS OLAP en AWS (Learner Lab)
# aerolinea-olap.rds  2026-09-03T02:55:51Z

=== v_ocupacion_por_ruta ===
par_ruta | ciudad_origen | ciudad_destino | anio | mes | temporada | pasajeros_confirmados |
vuelos | asientos_ofertados | pct_ocupacion
BOG-CLO | Bogota | Cali | 2026 | 9 | BAJA | 0 | 3 | 318 | 0.0
BOG-CTG | Bogota | Cartagena | 2026 | 9 | BAJA | 2 | 3 | 486 | 0.4
BOG-MDE | Bogota | Rionegro | 2026 | 9 | BAJA | 6 | 6 | 408 | 1.5
MDE-CTG | Rionegro | Cartagena | 2026 | 9 | BAJA | 1 | 2 | 136 | 0.7

=== v_ingresos_por_tarifa ===
clase_tarifa | cabina | ventana_compra | anio | mes | ingreso_confirmado | comision_pagada |
pasajeros | ingreso_promedio_pax
PROMO | ECONOMICA | anticipada | 2026 | 9 | 1940000.00 | 32760.00 | 9 | 215555.56

=== v_patrones_cancelacion ===
par_ruta | clase_tarifa | canal | anio | mes | temporada | segmentos_totales |
segmentos_cancelados | segmentos_expirados | pct_cancelacion | ingreso_perdido_cancelaciones
BOG-MDE | PROMO | DIRECTO | 2026 | 9 | BAJA | 5 | 0 | 0 | 0.0 | None
MDE-CTG | PROMO | DIRECTO | 2026 | 9 | BAJA | 2 | 1 | 0 | 50.0 | 236000.00
BOG-MDE | PROMO | AGENCIA | 2026 | 9 | BAJA | 1 | 0 | 0 | 0.0 | None
BOG-CLO | PROMO | AGENCIA | 2026 | 9 | BAJA | 2 | 1 | 0 | 50.0 | 204000.00
BOG-CTG | PROMO | DIRECTO | 2026 | 9 | BAJA | 2 | 0 | 0 | 0.0 | None
BOG-CTG | PROMO | AGENCIA | 2026 | 9 | BAJA | 1 | 0 | 0 | 0.0 | None
BOG-CLO | PROMO | DIRECTO | 2026 | 9 | BAJA | 1 | 1 | 0 | 100.0 | 204000.00
```

Apéndice — Registro de prompts (prompt log)

#	Tema	Instrucción del equipo (resumen)	Aporte del asistente	Qué corrigió / decidió el equipo
P 1	Encuadre	"Analizamos el enunciado. El escenario es una aerolínea regional, despliegue en AWS Academy. Queremos monolito, PostgreSQL y control de sobreventa con bloqueo.	Borrador de plan por fases y estructura de carpetas del repo.	El equipo fijó las decisiones de fondo (monolito, bloqueo pesimista, alcance mínimo) antes de pedir el plan.

		Propón un plan de trabajo por fases."		
P 2	Contexto y supuestos	"Redacta el documento de contexto y la lista de supuestos S1–S10 a partir de estas decisiones nuestras: hold de 15 min, asiento se asigna en check-in, vuelo programado ≠ instancia, itinerario como entidad propia, sin programa de viajero frecuente en el MVP…"	Redacción del documento y tabla de supuestos con "impacto si cambia".	El equipo definió cada supuesto y su justificación; ajustó la redacción de S4, S7 y S9.
P 3	Requisitos 5.1 / 5.2	"Con estos actores (pasajero, agente de agencia, staff de aeropuerto, admin) y estas reglas de negocio, arma la tabla de 18 FR. Para los NFR queremos: búsqueda p95 < 800 ms directos / < 1,5 s con escalas, invariante de no sobreventa, 99,9 % disponibilidad, sin almacenar PAN/CVV, auditoría append-only."	Tablas de FR y NFR en el formato pedido; propuso categorías y verificación de cada NFR.	El equipo fijó los valores concretos de las métricas y la prioridad de cada FR; revisó que cada NFR tuviera métrica + riesgo de negocio.
P 4	Modelo E-R 5.3	"Modela el E-R con notación Crow's Foot. Entidades mínimas + las que hagan falta. Decisiones nuestras: inventario por cabina (contador, no por silla), `VUELO_PROGRAMADO` vs `INSTANCIA_VUELO`, `ITINERARIO` entre `RESERVA` e `INSTANCIA_VUELO`, OLTP en 3FN."	Diccionario de datos de las 19 entidades y el diagrama `.drawio`.	El equipo revisó el diagrama: pidió reorganizarlo en 5 bandas por capa, en español, y con conectores ortogonales (sin líneas en diagonal ni cruces sobre cajas).
P 5	Arquitectura 5.4 / 5.5	"Redacta la arquitectura backend y frontend. Cada decisión debe citar el FR/NFR que la origina. Backend: monolito modular, `SELECT ... FOR UPDATE` con orden determinista, pago delegado a pasarela externa, agencias por la misma API. Frontend: SPA, manejo del 409 sin WebSocket, i18n/multi-moneda preparados."	Documentos `04` y `05` con las alternativas descartadas (bloqueo optimista, cola, SSR) argumentadas.	El equipo definió el mecanismo de concurrencia y la postura sobre WebSocket; ajustó la sección de canal de agencias.
P	Implementac	"Implementa el backend según	Código base	El equipo revisó que el

6	ión backend 5.6	5.4: FastAPI + PostgreSQL con el modelo 5.3, endpoints buscar/crear/consultar, y `crear_reserva` con el bloqueo pesimista. Necesitamos una prueba que demuestre que 20 peticiones simultáneas por el último asiento dan 1 reserva y 19 rechazos."	(SQLAlchemy 2, Alembic, servicios) y dos pruebas de concurrencia (capa de servicio con 20 hilos + API HTTP en vivo).	código reflejara 5.4; documentó las desviaciones D1–D6 en `docs/06` §5.6.3. Ninguna obligó a cambiar el modelo E-R. Ambas pruebas pasan: 1 × 201, 19 × 409, sin sobreventa.
P 7	Implementac ión AWS 5.7	"Diseña el OLAP en esquema estrella que responda ocupación por ruta / ingresos por tarifa / patrones de cancelación. El ETL Python: EXTRACT una consulta a nivel de segmento, TRANSFORM desnormaliza y deriva `par_ruta`/`temporada`/`ventana_compra`, LOAD upsert idempotente. Justifica cada servicio AWS con un FR/NFR y un pilar Well-Architected, y proyecta costos + escenario ×10."	`schema_olap.sql` (hecho + 5 dims + 3 vistas), `etl/etl.py`, tabla de servicios y proyección de costos.	El equipo decidió el grano del hecho, la frecuencia del ETL (diaria, no event-triggered) y por qué script Python y no Glue Job. Corrigió la vista `v_ocupacion_por_ruta` (el denominador salía inflado por repetición de filas → se reescribió con CTEs).
P 8	Despliegue en AWS	"Organiza el cdk con cada stack CDK en el Learner Lab: VPC por defecto, 2× RDS PostgreSQL cifradas, EC2 con la API, y el Glue Data Catalog con los 2 crawlers."	Stack CDK en TypeScript y script de despliegue.	El equipo resolvió las restricciones del Lab: sin `cdk bootstrap` (el Lab no deja crear roles IAM) → `cdk synth` + `aws cloudformation deploy`; usar `LabRole`/`LabInstanceProfile`; PostgreSQL 16 (no 16.4, no disponible); el SG de Glue debe abrir todos los puertos internamente. Corrigió el provisioning (cargaba el `.env` tarde; corría un `pytest` que borraba el esquema).
P 9	Frontend 5.5	"Ya que la arquitectura del frontend está diseñada, impleméntala: SPA React con las pantallas búsqueda → selección → confirmación →	Código de la SPA (React + Vite + router) y CORS en el backend.	El equipo decidió no hacer pantallas de pago/cancelación porque el backend del MVP no expone esos

		código y consultar reserva. Debe manejar el 409 y tener los textos externalizados."		endpoints (coherente con `docs/06` D6). Ajustes en `docs/05` §5.5.6.